



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ

KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Projekt dyplomowy

Wykorzystanie technik DevOps w implementacji automatycznego procesu CI/CD do uczenia i wersjonowania modeli sztucznej inteligencji: analiza utrzymania systemów wykorzystujących sztuczną inteligencję w porównaniu do metod manualnych

Using DevOps techniques to implement an automated CI/CD process for training and versioning AI models: an analysis of the maintenance of AI-based systems compared to manual methods

Autor: Rafał Zabłotni
Kierunek studiów: Inżynieria Obliczeniowa
Opiekun pracy: mgr inż. Piotr Hajder

Kraków, 2026

Spis treści

1	Wstęp	3
1.1	Geneza	3
1.2	Założenia	4
2	Wprowadzenie teoretyczne	5
2.1	DevOps	5
2.2	Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)	5
2.3	MLOps	6
2.4	Kontrola wersji — Git i GitHub	7
2.5	Pipeline — GitHub Actions	7
2.6	Wersjonowanie danych — DVC	8
2.7	Śledzenie eksperymentów — MLflow	8
2.8	Konteneryzacja — Docker	8
2.9	Orkiestrator Kubernetes — k3s	9
2.10	Infrastruktura jako kod — Terraform i Helm	9
2.11	Monitoring — Prometheus i Grafana	9
3	Architektura Systemu	10
3.1	Cele i ograniczenia projektowe	10
3.2	Struktura repozytoriów	11
3.3	Struktura architektury systemu	12
3.3.1	Użytkownik (Data Scientist)	12
3.3.2	Platforma kontroli wersji i automatyzacji (GitHub)	12
3.3.3	Środowisko chmurowe AWS	13
3.3.4	Środowisko wykonawcze na EC2	13
3.4	Kluczowe decyzje projektowe	14
4	Model Sztucznej Inteligencji	15
4.1	Charakterystyka problemu segmentacji semantycznej	15
4.2	Architektura modelu	15
4.3	Dane treningowe i przygotowanie zbioru danych	16
4.4	Proces uczenia i metryki jakości	17
4.5	Model jako artefakt w procesie CI/CD	17
5	Pipeline CI/CD	18
5.1	Hook pre-push	18
5.2	Struktura workflow GitHub Actions	19
5.3	Walidacja danych	20
5.4	Pipeline treningowy	21
5.5	Bramka jakości	23
6	Wdrożenie Systemu	25

6.1	Infrastruktura Terraform	26
6.2	Konfiguracja EC2 i k3s	27
6.3	Dostępne porty i usługi	29
6.4	Konfiguracja Helm	30
6.5	Stos monitorowania	32
6.6	Workflow użytkownika	33
6.6.1	Scenariusz 1: Ręczne wdrożenie i korzystanie z API (zalecany)	33
6.6.2	Scenariusz 2: Wdrożenie przez automatyczny pipeline	34
6.7	Dostępne operacje zarządzania wdrożeniem	34
7	Analiza Wyników i Działania Systemu	35
7.1	Cel analizy i kryteria oceny	35
7.2	Metodologia porównania	35
7.3	Czasy wykonania i nakład pracy człowieka	36
7.4	Jakość modeli i skuteczność bramki jakości	36
7.5	Koszty infrastruktury	40
7.6	Zużycie zasobów i działanie usługi	42
7.7	Ocena jakościowa procesu	43
7.7.1	Łatwość wdrożenia	43
7.7.2	Stabilność i podatność na błąd	43
7.7.3	Skalowalność	43
8	Podsumowanie i wnioski	44
8.1	Osiągnięcia projektu	44
8.2	Odpowiedź na pytania badawcze	44
8.3	Kluczowe wnioski z porównania	45
8.4	Ograniczenia badania	45
8.5	Wnioski końcowe	46
8.6	Wykorzystanie narzędzi sztucznej inteligencji	46
	Literatura	47

1 Wstęp

1.1 Geneza

Pomysł na realizację niniejszego projektu zrodził się podczas analizowania wysokopoziomowych testów modułowych. Wspomniane testy charakteryzują się wysokim poziomem abstrakcji oraz szerokim zakresem funkcjonalności, co skutkuje dużą liczbą wymaganych powtarzalnych ustawień. Na tej podstawie zaobserwowano interesującą tendencję polegającą na tym, że znaczna część kodu testowego okazywała się redundantna, a wiele mechanizmów było wielokrotnie powielanych poprzez kopiowanie oraz wprowadzanie drobnych modyfikacji do istniejących fragmentów kodu.

Naturalnym rozwiązaniem tego problemu mogłoby wydawać się wydzielanie powtarzalnych elementów do funkcji pomocniczych lub bibliotek testowych oraz utrzymywanie ich w uporządkowanej i współdzielonej formie. Podejście to jednak wiąże się z istotnymi trudnościami, zarówno natury organizacyjnej, jak i technicznej. W praktyce często prowadzi ono do nadmiernego rozrostu bibliotek pomocniczych, niejednoznacznych interfejsów oraz problemów z ich długoterminowym utrzymaniem.

W szczególności pojawiają się następujące problemy:

1. Jak określić moment, w którym dana funkcjonalność jest wystarczająco generyczna, aby mogła zostać wydzielona do wspólnej biblioteki?
2. Jak zminimalizować ryzyko tworzenia funkcji lokalnie w plikach testowych, które z czasem zaczynają być wykorzystywane przez inne testy, prowadząc do niejawnych zależności pomiędzy teoretycznie niezależnymi modułami?
3. Jak zapewnić spójny sposób tworzenia oraz utrzymania takich funkcji w projektach rozwijanych przez dziesiątki, a niekiedy setki programistów, o różnym poziomie doświadczenia i odmiennych nawykach pracy?

Jednocześnie, jak powszechnie wiadomo, modele sztucznej inteligencji uczą się na podstawie zbiorów danych treningowych, a następnie wykorzystują wyuczone wzorce do rozwiązywania nowych problemów. Obserwacja ta stała się punktem wyjścia do rozważenia możliwości zastosowania modeli sztucznej inteligencji (ang. Artificial Intelligence, AI) w obszarze automatycznego generowania wspólnych, powtarzalnych fragmentów kodu testowego na podstawie już istniejących przykładów.

Jednakże, aby taki system mógł zostać efektywnie wdrożony w praktyce, niezbędne jest zaprojektowanie i utrzymanie procesu ciągłej integracji i ciągłego dostarczania (ang. Continuous Integration / Continuous Delivery, CI/CD), zapewniającego automatyczne trenowanie modeli, ich ewaluację oraz kontrolowane wdrażanie w środowisku produkcyjnym. W przeciwnym razie konieczne byłoby wyznaczenie osoby odpowiedzialnej za manualne utrzymanie i nadzór.

W ten sposób wyłaniają się dwa kluczowe pytania stanowiące oś niniejszego projektu:

1. Czy stworzenie modelu AI umożliwiającego automatyczne generowanie fragmentów kodu testowego jest rozwiązaniem możliwym do realizacji oraz uzasadnionym ekonomicznie?
2. Czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymywania i rozwijania tego rozwiązania, czy też zasadne jest jego zautomatyzowanie z wykorzystaniem narzędzi DevOps oraz procesów CI/CD?

1.2 Założenia

Ze względu na złożoność problemu generowania kodu testowego oraz ograniczony czas realizacji projektu zdecydowano się na zastosowanie problemu zastępczego, umożliwiającego weryfikację postawionej hipotezy bez konieczności budowania zaawansowanego modelu językowego. Jako problem zastępczy wybrano segmentację semantyczną obrazów wodomierzy z wykorzystaniem architektury U-Net. Wybór ten jest uzasadniony, ponieważ modele segmentacyjne posiadają jednoznacznie zdefiniowane metryki jakości, takie jak współczynnik Dice (ang. Dice coefficient, miara nakładania masek) oraz IoU (ang. Intersection over Union, stosunek części wspólnej do sumy obszarów), charakteryzują się umiarkowanymi wymaganiami obliczeniowymi umożliwiającymi trening w ramach dostępnego budżetu, a także opierają się na dostępnych modelach wstępnie wytrenowanych, co pozwala na obiektywne porównanie kolejnych wersji.

Dla uproszczenia przyjęto założenie, że model sztucznej inteligencji dostarczany jest przez zewnętrzną firmę, natomiast zakres pracy obejmuje jego dalsze uczenie, ocenę jakości oraz wdrażanie. Takie podejście odzwierciedla realny scenariusz środowiska produkcyjnego, w którym zespół DevOps otrzymuje gotowy model i odpowiada za zarządzanie jego cyklem życia.

W tak zdefiniowanym kontekście drugie pytanie sformułowane w poprzedniej sekcji pozostaje aktualne: czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymywania i rozwijania tego rozwiązania, czy też zasadne jest jego zautomatyzowanie z wykorzystaniem narzędzi DevOps oraz procesów CI/CD? W podejściu manualnym proces ponownego uczenia i wdrażania modelu obejmuje ręczne przygotowanie danych, uruchamianie treningu, walidację wyników oraz publikację nowej wersji. Podejście to, mimo swojej prostoty koncepcyjnej, wiąże się z ryzykiem błędów ludzkich, trudnościami w zachowaniu powtarzalności eksperymentów oraz ograniczoną skalowalnością. Automatyzacja natomiast umożliwia jednoznaczne powiązanie wersji modelu z wersją danych treningowych, wprowadzenie obiektywnej bramki jakości oraz eliminację ręcznych etapów mogących prowadzić do niespójności.

W ramach niniejszej pracy opracowano kompletny, zautomatyzowany system CI/CD dla modeli sztucznej inteligencji. System obejmuje wersjonowanie danych treningowych z wykorzystaniem narzędzia DVC, śledzenie eksperymentów i rejestr modeli (MLflow), automatyczny potok (ang. pipeline) treningowy uruchamiany w GitHub Actions, mechanizm bramki jakości porównujący nowy model z aktualną wersją produkcyjną oraz wdrożenie modelu w środowisku chmurowym za pomocą lekkiej dystrybucji Kubernetes (k3s), uruchomionej na instancji Amazon EC2 (ang. Elastic Compute Cloud, Elastyczna Chmura Obliczeniowa) wraz z mechanizmami monitoringu (Prometheus, Grafana). Całość zaprojektowano w taki sposób, aby dodanie nowych danych treningowych wymagało jedynie wykonania polecenia `git push`, a pozostałe etapy procesu przebiegały automatycznie. Tak zdefiniowany zakres umożliwia bezpośrednie porównanie kosztów i nakładu pracy podejścia manualnego oraz zautomatyzowanego, co stanowi główny cel analityczny pracy.

2 Wprowadzenie teoretyczne

Realizacja projektu opisanego w niniejszej pracy opiera się na zestawie technologii oraz koncepcji z zakresu inżynierii oprogramowania, uczenia maszynowego i infrastruktury chmurowej. W rozdziale przedstawiono teoretyczne podstawy tych zagadnień, stanowiące punkt odniesienia dla decyzji projektowych omówionych w kolejnych częściach pracy. W pierwszej kolejności zaprezentowano DevOps oraz mechanizmy CI/CD, następnie podejście MLOps jako ich rozszerzenie na obszar modeli uczenia maszynowego. W dalszej części omówiono narzędzia wykorzystane w projekcie, w tym Git i GitHub, GitHub Actions, MLflow, DVC, Docker, Kubernetes (k3s), Terraform wraz z Helm, oraz systemy monitoringu Prometheus i Grafana.

2.1 DevOps

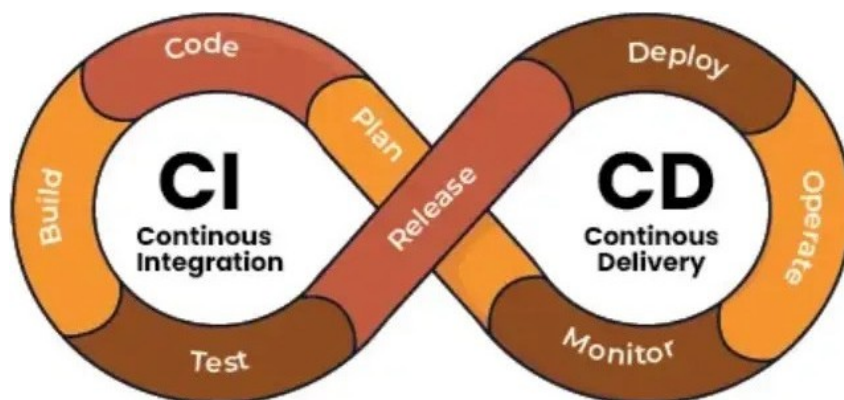
DevOps to zbiór praktyk, kultury organizacyjnej i narzędzi, których celem jest skrócenie cyklu życia oprogramowania przy jednoczesnym zapewnieniu wysokiej jakości dostarczanych produktów. Termin pochodzi od połączenia słów *Development* i *Operations*, co odzwierciedla dążenie do przełamania tradycyjnej bariery między zespołami deweloperskimi a operacyjnymi.

Kluczowym elementem filozofii DevOps jest pętla ciągłego doskonalenia, obejmująca planowanie, kodowanie, budowanie, testowanie, wdrażanie, uruchamianie oraz monitorowanie. Każdy obrót pętli dostarcza informacje zwrotne, które pozwalają szybciej reagować na błędy i wymagania użytkowników. W kontekście niniejszej pracy DevOps stanowi ramy koncepcyjne, w których osadzony jest cały projekt [12].

2.2 Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)

Ciągła Integracja (ang. *Continuous Integration*, CI) polega na systematycznym i automatycznym scalaniu oraz weryfikacji zmian w kodzie repozytorium. Każda zmiana inicjuje uruchomienie zdefiniowanego zestawu testów oraz mechanizmów kontroli jakości, co pozwala wykrywać błędy integracyjne bezpośrednio po ich wprowadzeniu.

Ciągłe Dostarczanie (ang. *Continuous Delivery*, CD) rozszerza podejście CI o automatyczne przygotowanie artefaktów gotowych do wdrożenia, takich jak skompilowana aplikacja lub obraz kontenera. Ciągłe Wdrażanie (ang. *Continuous Deployment*) stanowi kolejny etap automatyzacji procesu, w którym każda zmiana spełniająca określone kryteria jakości jest automatycznie wdrażana do środowiska produkcyjnego.



Rys. 1 Przykładowy schemat procesu CI/CD [14]

Typowy potok CI/CD składa się z następujących etapów:

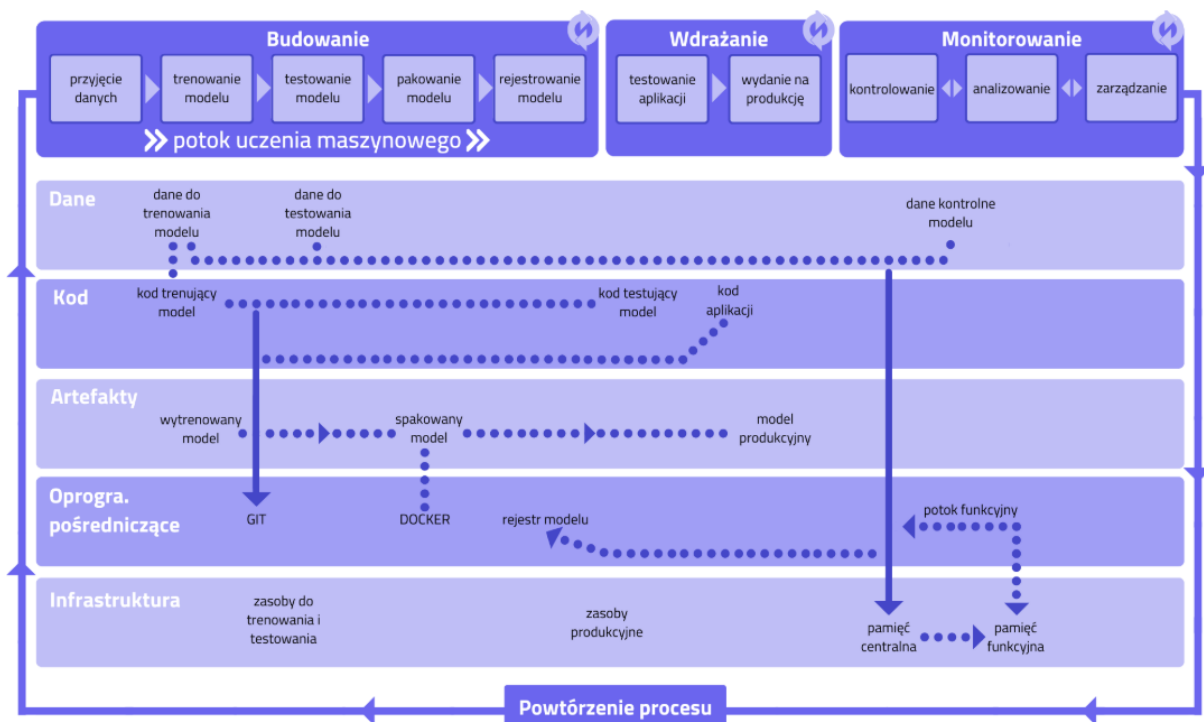
1. weryfikacja kodu (statyczna analiza kodu, tzw. linting, oraz testy jednostkowe),
2. budowanie artefaktów (np. kompilacja aplikacji lub budowa obrazu Docker),
3. testy integracyjne i akceptacyjne,
4. wdrożenie na środowisko docelowe,
5. weryfikacja po wdrożeniu (tzw. testy typu smoke).

Należy podkreślić, że szczegółowa struktura pipeline'u może różnić się w zależności od charakteru projektu, przyjętej architektury systemu oraz poziomu dojrzałości procesów wytwórczych w organizacji.

2.3 MLOps

MLOps (ang. *Machine Learning Operations*) to dyscyplina stosująca zasady DevOps do specyfikacji cyklu życia modeli uczenia maszynowego. W odróżnieniu od klasycznego oprogramowania, gdzie artefaktem jest kod, w ML artefaktem jest model czyli wynik trenowania na konkretnym zbiorze danych. To wprowadza kilka dodatkowych wyzwań:

- Dane treningowe zmieniają się niezależnie od kodu.
- Wyniki trenowania są niedeterministyczne.
- Jakość modelu musi być mierzona obiektywnie przed każdym wdrożeniem.
- Modele degradują się w czasie (tzw. *data drift*).



Rys. 2 Przykład flow MLOps [2]

MLflow obejmuje cały cykl życia modelu uczenia maszynowego, począwszy od rejestrowania eksperymentów, parametrów i metryk podczas treningu, poprzez wersjonowanie modeli, aż po zarządzanie ich etapami w rejestrze modeli. Monitoring działającej aplikacji produkcyjnej realizowany jest z wykorzystaniem dedykowanych narzędzi takich jak Prometheus i Grafana.

2.4 Kontrola wersji — Git i GitHub

Git to rozproszony system kontroli wersji, który stanowi standard w nowoczesnym procesie wytwarzania oprogramowania [4]. Przechowuje pełną historię zmian w postaci kolejnych zapisów stanu projektu (commitów), umożliwiając równoległą pracę na gałęziach (ang. *branches*), scalanie zmian oraz cofanie się do dowolnego punktu w historii projektu.

Kluczowe koncepcje systemu Git:

- **Commit** — zapis stanu repozytorium w danym momencie wraz z metadanymi (autor, czas, opis zmiany),
- **Branch** — niezależna linia rozwoju umożliwiająca pracę nad nową funkcjonalnością bez wpływu na gałąź główną,
- **Merge / Pull Request** — proces scalania zmian z jednej gałęzi do drugiej, zazwyczaj poprzedzony przeglądem kodu,
- **Hook** — skrypt wykonywany automatycznie w odpowiedzi na określone zdarzenie w systemie Git (np. *pre-push*).

GitHub to platforma hostingowa dla repozytoriów Git, rozszerzona o funkcje wspierające współpracę zespołową, takie jak mechanizm pull requestów, przeglądy kodu, zarządzanie zadaniami oraz zintegrowane mechanizmy CI/CD w postaci GitHub Actions. W kontekście niniejszego projektu GitHub pełni rolę centralnego punktu integracji, przez który przechodzą wszystkie zmiany. Każda z nich może automatycznie inicjować kolejne etapy potoku CI/CD.

2.5 Pipeline — GitHub Actions

GitHub Actions to platforma automatyzacji wbudowana w GitHub, pozwalająca definiować przepływy pracy (ang. *workflows*) jako pliki YAML przechowywane bezpośrednio w repozytorium [6]. Workflow uruchamiane są w odpowiedzi na zdarzenia: push, pull request, harmonogram (cron) lub wywołanie ręczne.

Podstawowe pojęcia:

- **Workflow** — zestaw automatycznych zadań uruchamianych w ramach GitHub Actions, zdefiniowany w pliku `.github/workflows/*.yaml`, opisujący m.in. kiedy proces ma się uruchomić (triggery), jakie środowisko wykorzystać oraz jakie kroki wykonać.
- **Job** — jednostka wykonania złożona z kroków (steps); domyślnie uruchamiana równoległe z innymi jobami,
- **Step** — pojedyncza operacja w ramach joba: wywołanie skryptu lub gotowej akcji,
- **Runner** — maszyna wykonująca joba: może być zarządzana przez GitHub (hosted) lub przez użytkownika (self-hosted),
- **Artefakt** — plik lub katalog przekazywany między jobami albo pobierany po zakończeniu workflow,
- **Secrets** — zaszyfrowane zmienne środowiskowe przechowywane na poziomie repozytorium. Umożliwiają bezpieczne używanie poświadczeń i kluczy API do workflow.

Kluczową cechą GitHub Actions jest możliwość użycia **self-hosted runners**, czyli maszyn zarządzanych przez użytkownika. W niniejszym projekcie mechanizm ten wykorzystywany jest na etapie wdrożenia modelu, gdzie wymagany jest bezpośredni dostęp do lokalnej infrastruktury (Docker, klaster k3s) uruchomionej na instancji EC2.

2.6 Wersjonowanie danych — DVC

DVC (ang. *Data Version Control*) to narzędzie umożliwiające wersjonowanie dużych plików binarnych (obrazów, wag modeli, zbiorów danych) w sposób spójny z Git. Zamiast przechowywać dane w repozytorium Git (co jest niepraktyczne przy dużych plikach), DVC przechowuje jedynie lekkie pliki metadanych (`.dvc`), a rzeczywiste dane trafiają do zewnętrznego magazynu, którym w naszym przypadku jest Amazon S3 [9].

Mechanizm działania jest analogiczny do systemu Git: każda wersja danych posiada własny identyfikator w postaci skrótu (hash), a po przełączeniu repozytorium na wybraną wersję możliwe jest pobranie odpowiadających jej danych za pomocą polecenia `dvc pull`. Umożliwia to odtworzenie dokładnego eksperymentu, rozumianego jako ta sama wersja kodu, danych i konfiguracji.

2.7 Śledzenie eksperymentów — MLflow

MLflow to platforma z publicznie dostępnym kodem źródłowym (ang. open-source) służąca do zarządzania cyklem życia modeli uczenia maszynowego. Składa się z czterech głównych komponentów:

- **Tracking** — rejestrowanie parametrów, metryk i artefaktów każdego uruchomienia eksperymentu,
- **Projects** — standaryzacja i reprodukowalność kodu ML,
- **Models** — zunifikowany format pakowania modeli,
- **Model Registry** — centralny rejestr wersji modeli z etapami cyklu życia (Staging, Production, Archived).

W kontekście projektu Model Registry pełni kluczową rolę w mechanizmie bramki jakości; szczegółowy opis implementacji znajduje się w podrozdziale 5.5 [15].

2.8 Konteneryzacja — Docker

Docker to platforma do tworzenia, dystrybucji i uruchamiania aplikacji w kontenerach [13]. Kontener to izolowane środowisko uruchomieniowe zawierające aplikację wraz ze wszystkimi jej zależnościami: bibliotekami, interpreterem, zmiennymi środowiskowymi. Dzięki temu aplikacja działa identycznie na maszynie dewelopera, serwerze testowym i produkcji, eliminując problem „u mnie działa”.

Kluczowe pojęcia:

- **Obraz (image)** — niezmienny szablon środowiska definiowany plikiem `Dockerfile`,
- **Kontener** — działająca instancja obrazu: uruchomiona w kontenerze, izolowana od systemu hosta i innych kontenerów
- **Rejestr (registry)** — repozytorium obrazów (np. Docker Hub, Amazon ECR).

W projekcie Docker służy do pakowania API modelu oraz do zapewnienia powtarzalności środowiska treningowego. Obraz z modelem jest budowany automatycznie w pipeline po zatwierdzeniu nowej wersji modelu.

2.9 Orkiestrator Kubernetes — k3s

Kubernetes (K8s) to system do orkiestracji kontenerów, który automatyzuje ich wdrażanie, skalowanie i zarządzanie [3]. Zamiast uruchamiać kontenery ręcznie na konkretnych maszynach, Kubernetes przyjmuje deklaracyjny opis pożądanego stanu (ile replik, jakie zasoby, jak reagować na awarie) i utrzymuje ten stan niezależnie od zdarzeń infrastrukturalnych.

k3s to uproszczona dystrybucja Kubernetes opracowana przez Rancher Labs. Charakteryzuje się zmniejszonymi wymaganiami sprzętowymi (minimalna instalacja może działać na instancji z 2 GB RAM) oraz uproszczonym procesem instalacji. Jest przeznaczona dla środowisk brzegowych, systemów Internetu Rzeczy (ang. Internet of Things, IoT) oraz małych klastrów, gdzie istotne są ograniczone zasoby sprzętowe i niskie koszty utrzymania.

W niniejszym projekcie klaster k3s został uruchomiony na pojedynczej instancji Amazon EC2, stanowiącej środowisko wykonawcze dla komponentów systemu. Takie podejście umożliwia zachowanie elastyczności infrastruktury przy jednoczesnym ograniczeniu złożoności środowiska testowego.

2.10 Infrastruktura jako kod — Terraform i Helm

Infrastruktura jako kod (ang. *Infrastructure as Code*, IaC) to praktyka definiowania i zarządzania zasobami infrastrukturalnymi za pomocą plików konfiguracyjnych przechowywanych w systemie kontroli wersji, zamiast ręcznego konfigurowania przez interfejs graficzny lub CLI [7]. Dzięki temu środowisko jest powtarzalne, audytowalne i odtwarzalne dowolnym momencie.

Terraform to narzędzie IaC umożliwiające tworzenie, modyfikowanie i niszczenie zasobów chmurowych w deklaracyjnym języku HCL (ang. *HashiCorp Configuration Language*). Terraform utrzymuje plik stanu (`terraform.tfstate`) opisujący aktualną konfigurację infrastruktury, co pozwala wykrywać i stosować wyłącznie te zmiany, które są niezbędne. W projekcie Terraform zarządza zasobami AWS: siecią VPC, instancją EC2, zasobnikami S3 i rejestrem ECR.

Helm to menedżer pakietów dla Kubernetes, analogiczny do apt czy pip w świecie aplikacji [8]. Podstawową jednostką dystrybucyjną w Helm jest tzw. *chart*, czyli pakiet zawierający szablony zasobów Kubernetes parametryzowane wartościami z pliku `values.yaml`. Umożliwia to instalację, aktualizację i usuwanie złożonych aplikacji wieloskładnikowych jednym poleceniem, z pełną historią wdrożeń i możliwością cofnięcia (rollback). W projekcie Helm zarządza wdrożeniem API modelu oraz stosu monitorującego na klastrze k3s.

2.11 Monitoring — Prometheus i Grafana

Prometheus to system monitorowania i alertowania oparty na modelu *pull* [16]. Regularnie pobiera (ang. *scrape*) metryki z endpointów HTTP udostępnianych przez monitorowane usługi i przechowuje je w wbudowanej bazie szeregów czasowych. Metryki opisywane są etykietami (ang. *labels*), co pozwala na elastyczne filtrowanie i agregację.

Grafana to platforma wizualizacji danych, która integruje się z wieloma źródłami, w tym z Prometheus. Pozwala tworzyć interaktywne dashboardy z wykresami, wskaźnikami i alertami. W projekcie Grafana prezentuje metryki działającego API modelu: liczbę predykcji, czasy odpowiedzi, zużycie zasobów i liczbę błędów. Monitoring działa jako opcjonalny stos uruchamiany na tym samym klastrze k3s co API modelu [11].

3 Architektura Systemu

Niniejszy rozdział przedstawia zaprojektowaną architekturę systemu, obejmującą wybrane komponenty, ich wzajemne powiązania oraz przesłanki kluczowych decyzji projektowych. Najbardziej rozbudowane funkcjonalności zostały opisane w odrębnych rozdziałach.

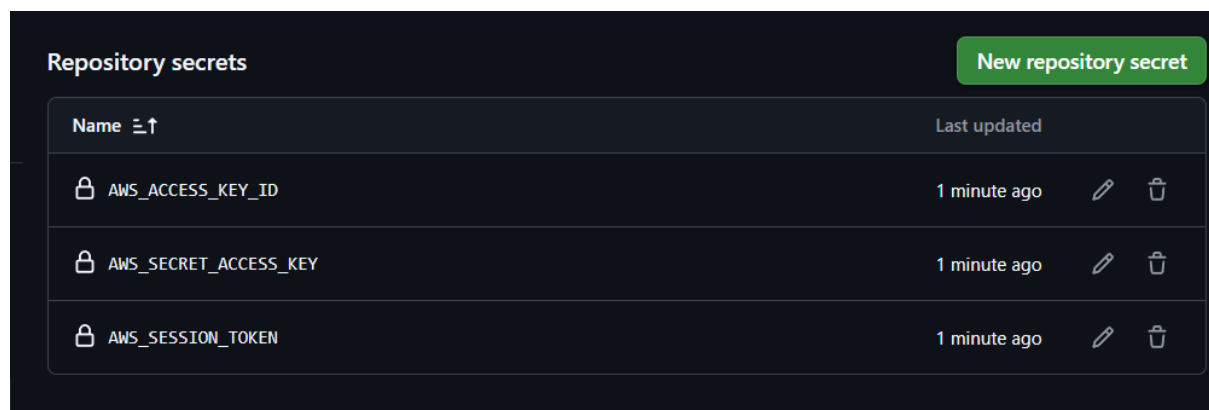
3.1 Cele i ograniczenia projektowe

Projektując system, kierowano się pięcioma celami:

1. **Pełna automatyzacja** — użytkownik jedynie będzie używać `git push` lub uruchamiać aplikacje; walidacja, trening, ocena jakości i wdrożenie odbywają się bez interwencji.
2. **Reprodukowalność** — każda wersja modelu musi być powiązana z wersją kodu, wersją danych i zapisanymi metrykami.
3. **Kontrola kosztów** — projekt realizowany w ramach konta AWS Academy z budżetem 50 USD, dlatego zasoby uruchamiane są wyłącznie w momencie ich faktycznej potrzeby.
4. **Kontrola czasu** — całkowity pipeline nie powinien przekraczać 4-godzinnego okna sesji laboratorium AWS. Szacowany czas samego treningu wynosił ok. 3 h.
5. **Porównywalność** — system musi być zbudowany w sposób umożliwiający zebranie wymierzanych danych w celu porównania z podejściem manualnym.

Dodatkowym ograniczeniem środowiska AWS Academy był zakres uprawnień przydzielonej roli laboratoryjnej, który nie umożliwiał bezpośredniej konfiguracji federacji tożsamości OIDC (ang. OpenID Connect, mechanizm federacyjnego uwierzytelniania) dla zewnętrznych systemów CI/CD, takich jak GitHub Actions. W ramach przyznanych uprawnień IAM (ang. Identity and Access Management, system zarządzania tożsamością i dostępem) mechanizm ten nie był dostępny do konfiguracji. Dozwolone było natomiast korzystanie z ról IAM przypisanych do instancji EC2.

W konsekwencji wykorzystano tymczasowe klucze sesyjne generowane przy każdym uruchomieniu środowiska laboratoryjnego. Klucze te były przechowywane jako zaszyfrowane sekrety repozytorium GitHub i wstrzykiwane do zmiennych środowiskowych workflow podczas jego wykonywania [6]. Ze względu na czasowy charakter sesji laboratoryjnych (maksymalnie kilka godzin), sekrety były rotowane ręcznie przy każdym odnowieniu środowiska, co stanowiło akceptowalny kompromis w kontekście projektu badawczego (rysunek 3).



Rys. 3 Konfiguracja sekretów GitHub Actions przechowujących tymczasowe poświadczenia AWS

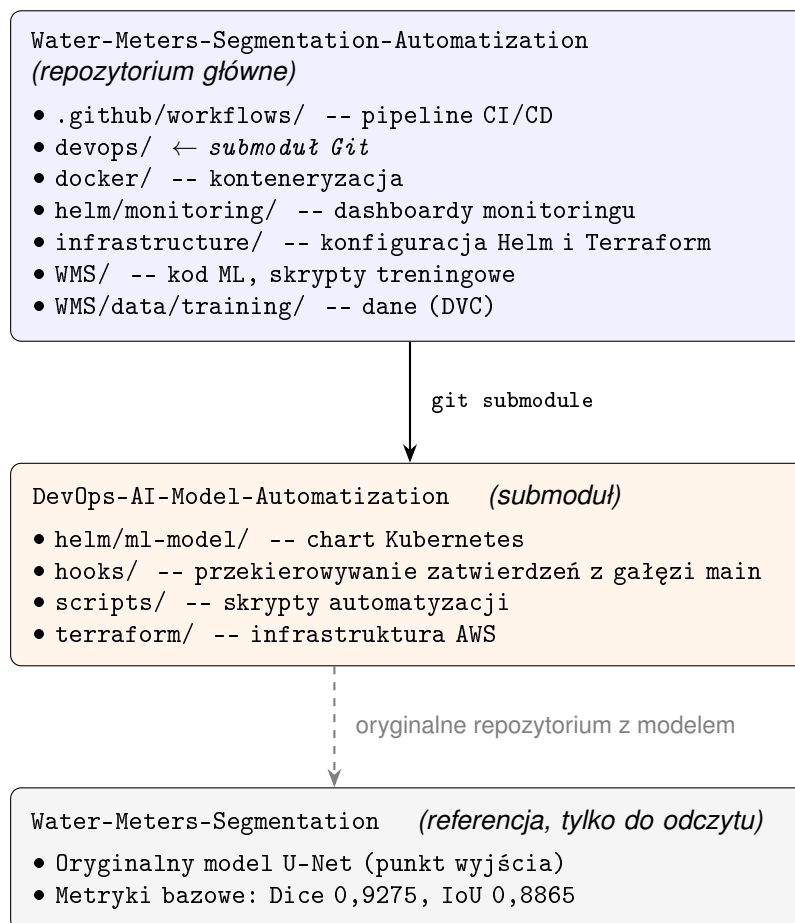
3.2 Struktura repozytoriów

Projekt korzysta z dwóch repozytoriów Git połączonych mechanizmem submodułów:

- **Water-Meters-Segmentation-Automatization** — repozytorium główne, zawiera kod ML, metadane danych treningowych śledzone przez DVC, workflow GitHub Actions, konfigurację Docker oraz skrypty pomocnicze. Obsługuje wyzwalanie całego procesu CI/CD.
- **DevOps-AI-Model-Automatization** — repozytorium infrastruktury, dołączone jako submoduł pod ścieżką `devops/`. Zawiera moduły Terraform, chart Helm modelu ML, skrypty konfiguracyjne EC2.
- **Water-Meters-Segmentation** — oryginalne repozytorium modelu U-Net. Służy wyłącznie jako punkt odniesienia maksymalnie wytrenowanego modelu (Dice 0,9275, IoU 0,8865).

Podział na dwa repozytoria wynika z odmiennej dynamiki zmian: kod ML oraz dane treningowe aktualizowane są często i wyzwalają automatyczne procesy CI/CD, natomiast definicja infrastruktury zmienia się rzadziej i jest modyfikowana w sposób kontrolowany. Zastosowanie mechanizmu submodułów pozwala powiązać konkretną wersję infrastruktury z konkretną wersją kodu aplikacji, zapewniając jednoznaczną zgodność środowiska wdrożeniowego.

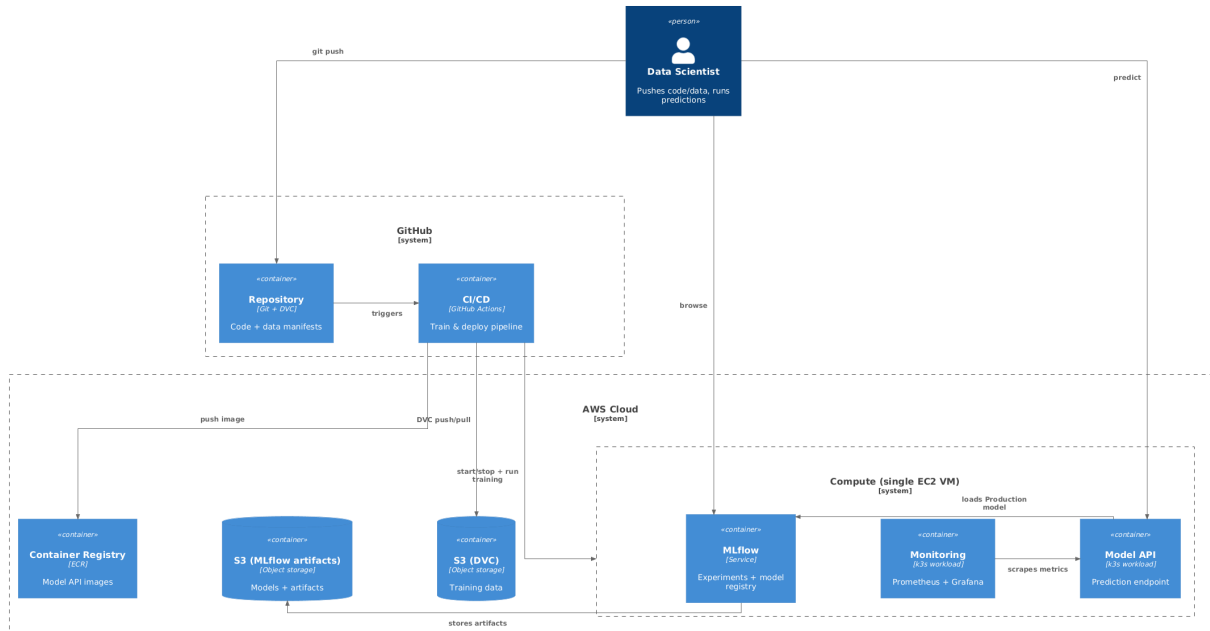
Katalog `infrastructure/` w repozytorium głównym zawiera jedynie wartości konfiguracyjne specyficzne dla tej instalacji (`terraform.tfvars`, `helm-values.yaml`), które nadpisują szablony infrastruktury z submodułu. Szablony pozostają w repozytorium infrastruktury, natomiast parametryzacja wdrożenia wersjonowana jest razem z kodem aplikacji.



Rys. 4 Struktura repozytoriów projektu

3.3 Struktura architektury systemu

Architektura systemu została zaprezentowana w modelu C4 (rysunek 5) i uporządkowana w cztery logiczne obszary odpowiadające granicom przedstawionym na diagramie: użytkownik, platforma GitHub, środowisko chmurowe AWS oraz środowisko wykonawcze na instancji EC2.



Rys. 5 Architektura systemu w modelu C4. Główne granice systemowe i komponenty.

3.3.1 Użytkownik (Data Scientist)

Punktem wejścia do systemu jest lokalna maszyna użytkownika (Data Scientist). Interakcja z systemem ogranicza się do dwóch typów operacji:

- dodawania nowych danych treningowych poprzez `git commit` i `git push`,
- wykonywania predykcji poprzez uruchomienie skryptu `predicts.py`.

Zainstalowany hook `Git pre-push` [4] automatycznie wykrywa `push` zawierający dane treningowe i przekierowuje go na gałąź `data/TIMESTAMP`. Mechanizm ten inicjuje pipeline treningowy bez potrzeby wykonywania dodatkowych czynności przez użytkownika.

Użytkownik ma również dostęp do interfejsu MLflow oraz endpointu predykcyjnego modelu wdrożonego w chmurze.

3.3.2 Platforma kontroli wersji i automatyzacji (GitHub)

Centralnym punktem systemu jest platforma GitHub, łącząca kontrolę wersji kodu z automatyzacją procesu trenowania i wdrażania modelu. Repozytorium Git [4] przechowuje kod modelu, definicje workflow GitHub Actions, konfigurację wdrożenia (Helm) oraz pliki `.dvc`: lekkie meta-dane wskazujące na dane treningowe przechowywane w Amazon S3. Mechanizm DVC opisany jest szczegółowo w podrozdziale 2.6.

GitHub Actions [6] odpowiada za pełny cykl życia modelu (Workflow):

1. wykrywa pojawienie się gałęzi danych,
2. pobiera historyczne dane z S3,
3. scala je z nowymi plikami,
4. uruchamia walidację,
5. startuje instancję EC2,
6. przeprowadza trening,
7. ocenia model przez bramkę jakości,
8. w przypadku poprawy wdraża nową wersję i scala zmiany z gałęzią `main`.

W trakcie wdrożenia obraz kontenera API budowany jest i publikowany do rejestru Amazon ECR, skąd następnie jest pobierany przez środowisko wykonawcze.

3.3.3 Środowisko chmurowe AWS

Granica AWS obejmuje zasoby infrastrukturalne tworzone deklaratywnie przy użyciu Terraform [7]. Kluczowe komponenty to:

- instancja EC2 `t3.large` z przypisanym Elastic IP,
- dwa zasobniki Amazon S3: jeden dla danych DVC, drugi dla artefaktów MLflow,
- rejestr kontenerów Amazon ECR.

Zasobnik S3 dla DVC przechowuje wersjonowane dane treningowe, natomiast drugi zasobnik przeznaczony jest na artefakty MLflow (modele, metryki, pliki pośrednie). Rejestr ECR pełni rolę pośrednika między procesem budowania obrazu w GitHub Actions a jego uruchomieniem w środowisku produkcyjnym.

Instancja EC2 pełni podwójną rolę: podczas treningu hostuje serwer MLflow, natomiast na etapie wdrożenia działa jako self-hosted runner GitHub Actions zainstalowany jako usługa `systemd`, odpowiedzialna za budowanie obrazu Docker oraz aktualizację klastra k3s.

3.3.4 Środowisko wykonawcze na EC2

Na instancji EC2 działają dwa typy komponentów: usługi systemowe oraz workloady uruchamiane w klastrze k3s [17].

MLflow działa jako usługa systemowa (`systemd`) bezpośrednio na systemie operacyjnym instancji. Odpowiada za rejestrowanie eksperymentów, przechowywanie metryk oraz zarządzanie rejestrem modeli [15]. Artefakty zapisywane są w dedykowanym zasobniku S3.

Wewnątrz lekkiego klastra k3s uruchomione są:

- API modelu (FastAPI + PyTorch),
- stos monitoringu (Prometheus + Grafana).

API modelu udostępnia endpoint `/predict`. Przy starcie kontenera model pobierany jest z rejestru MLflow w stadium Production, co rozdziela wersję obrazu kontenera od wersji samego modelu.

Prometheus zbiera metryki udostępniane przez API, a Grafana prezentuje je w postaci dashboardów. Konteneryzacja przy użyciu Docker [13] zapewnia powtarzalność środowiska uruchomieniowego niezależnie od cyklu start/stop instancji.

3.4 Kluczowe decyzje projektowe

Tabela 1 zestawia najważniejsze decyzje projektowe podjęte podczas budowy systemu, wraz z uzasadnieniem wyboru i odrzuconą alternatywą.

Tabela 1 Kluczowe decyzje architektoniczne

Decyzja	Wybór	Uzasadnienie
Orchestracja kontenerów	k3s na EC2	k3s jest w zupełności wystarczający dla jednego węzła i wielokrotnie tańszy od EKS (\$73/mies).
Backend MLflow	SQLite	Instancja RDS db.t3.large kosztuje ok. \$153/mies. SQLite jest wystarczający dla małego projektu, a metadane przechowywane są lokalnie na EC2.
Dostęp do API	NodePort	Load Balancer kosztuje ok. \$22/mies. Publiczny adres IP EC2 z Elastic IP jest wystarczający.
Typ instancji EC2	t3.large (8 GB RAM)	Na t3.small i t3.medium pobieranie obrazu PyTorch (8,69 GB) kończyło się awarią OOM.
Stop/start EC2	Automatyczny po każdym pipeline	EC2 działa tylko ok. 3–4 h na jedno uruchomienie. Daje to oszczędność ok. 80% kosztów w porównaniu z ciągłym działaniem.
Dane treningowe w Git	Tylko metadane DVC	Duże pliki binarne spowalniają Git i przekraczają jego limity rozmiaru. S3 jest tańszy i skalowalny.
Trening przed PR	Trening → PR (jeśli lepszy)	GitHub nie uruchamia workflow wywołanych przez PR tworzony automatycznie z GITHUB_TOKEN. Uruchomienie treningu przed utworzeniem PR eliminuje tę zależność.
Baseline bramki jakości	Dynamiczny z MLflow	Stała bramka wartości stawała by się nieaktualna po każdym treningu. MLflow zawsze przechowuje aktualne metryki aktualnego modelu Production.
IAM dla GitHub Actions	Tymczasowe klucze sesji	Rola laboratoryjna AWS Academy nie posiadała uprawnień do konfiguracji OIDC dla zewnętrznych systemów CI/CD. Dozwolone było korzystanie z ról IAM przypisanych do instancji EC2. Klucze sesyjne są rotowane ręcznie przy każdej sesji.
Artefakty MLflow	Amazon S3	Lokalny dysk EC2 jest ulotny. S3 zapewnia trwałość modeli i logów niezależnie od cyklu stop/start.

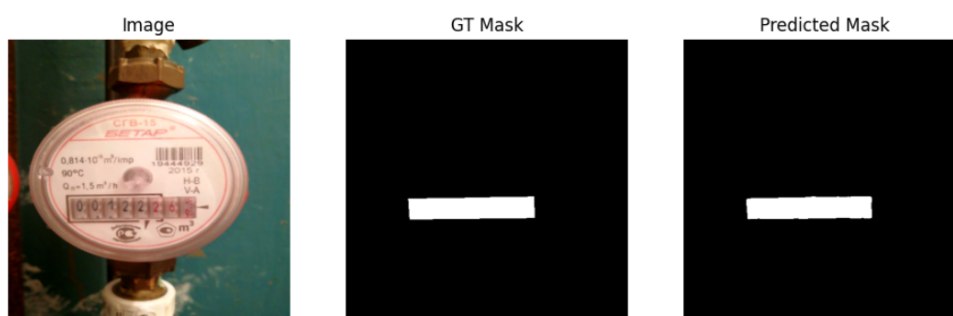
4 Model Sztucznej Inteligencji

4.1 Charakterystyka problemu segmentacji semantycznej

Model sztucznej inteligencji stanowiący podstawę niniejszego projektu został zaprojektowany do rozwiązywania zadania **segmentacji semantycznej obrazów wodomierzy**. Celem segmentacji jest przypisanie każdemu pikselowi obrazu etykiety określającej jego przynależność do jednej z **dwóch klas**: obszaru tarczy/licznika (foreground) oraz tła (background), co odpowiada binarnej masce wyjściowej.

Problem ten ma istotne znaczenie praktyczne w kontekście automatyzacji odczytu wskazań wodomierzy, gdzie poprawne wydzielenie obszaru tarczy stanowi warunek konieczny dla dalszych etapów przetwarzania obrazu, takich jak rozpoznawanie cyfr lub analiza wskazań mechanicznych. W przeciwieństwie do klasycznej klasyfikacji obrazów, segmentacja semantyczna wymaga zachowania informacji przestrzennej oraz precyzyjnego odwzorowania granic obiektu, co znacząco zwiększa złożoność problemu.

Dodatkowym wyzwaniem są zmienne warunki akwizycji danych, obejmujące różnice w oświetleniu, kącie wykonania zdjęcia, jakości obrazu oraz stopniu zabrudzenia lub zużycia urządzeń pomiarowych. Czynniki te powodują, że klasyczne algorytmy przetwarzania obrazu okazują się niewystarczające, co uzasadnia zastosowanie metod opartych na uczeniu głębokim.



Rys. 6 Przykład predykcji modelu segmentacji semantycznej

4.2 Architektura modelu

W projekcie wykorzystano konwolucyjną sieć neuronową opartą na architekturze **U-Net**, powszechnie stosowaną w zadaniach segmentacji semantycznej obrazów. Architektura ta charakteryzuje się symetryczną budową typu encoder-decoder, umożliwiającą jednocześnie wydobywanie cech semantycznych oraz zachowanie informacji przestrzennej.

Część kodująca (encoder) odpowiada za stopniowe przekształcanie obrazu wejściowego do reprezentacji o coraz wyższym poziomie abstrakcji poprzez sekwencję operacji konwolucji oraz redukcji rozdzielczości. Część dekodująca (decoder) realizuje proces rekonstrukcji obrazu wyjściowego, przywracając jego pierwotną rozdzielczość i generując maskę segmentacyjną.

Kluczowym elementem architektury są tzw. *skip connections*, umożliwiające bezpośrednie przekazywanie map cech pomiędzy odpowiadającymi sobie poziomami enkodera i dekodera. Mechanizm ten pozwala ograniczyć utratę informacji lokalnej oraz poprawić jakość segmentacji, szczególnie w obszarach granicznych pomiędzy klasami.

Zastosowana implementacja architektury U-Net została dostosowana do specyfiki problemu, w tym rozdzielczości obrazów wejściowych oraz liczby klas segmentacyjnych. Model generuje maskę wyjściową o tej samej rozdzielczości co obraz wejściowy, co ułatwia dalsze etapy przetwarzania oraz ocenę jakości predykcji.

Rys. 7 Schemat architektury U-Net [5]

Model został wytrenowany na zbiorze danych składającym się z obrazów wodomierzy oraz odpowiadających im masek. Każda próbka zawiera obraz wejściowy w postaci fotografii oraz maskę określającą przynależność poszczególnych pikseli do zdefiniowanych klas.

Zastosowanie augmentacji pozwala ograniczyć ryzyko przeuczenia modelu oraz poprawić jego zdolność do generalizacji na dane niewidziane w procesie uczenia. Istotnym aspektem projektu jest fakt, że zbiór danych oraz jego kolejne wersje podlegają wersjonowaniu, co umożliwia jednoznaczne powiązanie konkretnej wersji modelu z użytymi danymi treningowymi.

4.4 Proces uczenia i metryki jakości

Proces uczenia modelu realizowany jest w sposób nadzorowany, z wykorzystaniem funkcji straty dostosowanej do problemu segmentacji semantycznej. Jako główne metryki oceny jakości modelu zastosowano **współczynnik Dice** oraz **Intersection over Union (IoU)**, które są standardowymi miarami stosowanymi w zadaniach segmentacyjnych.

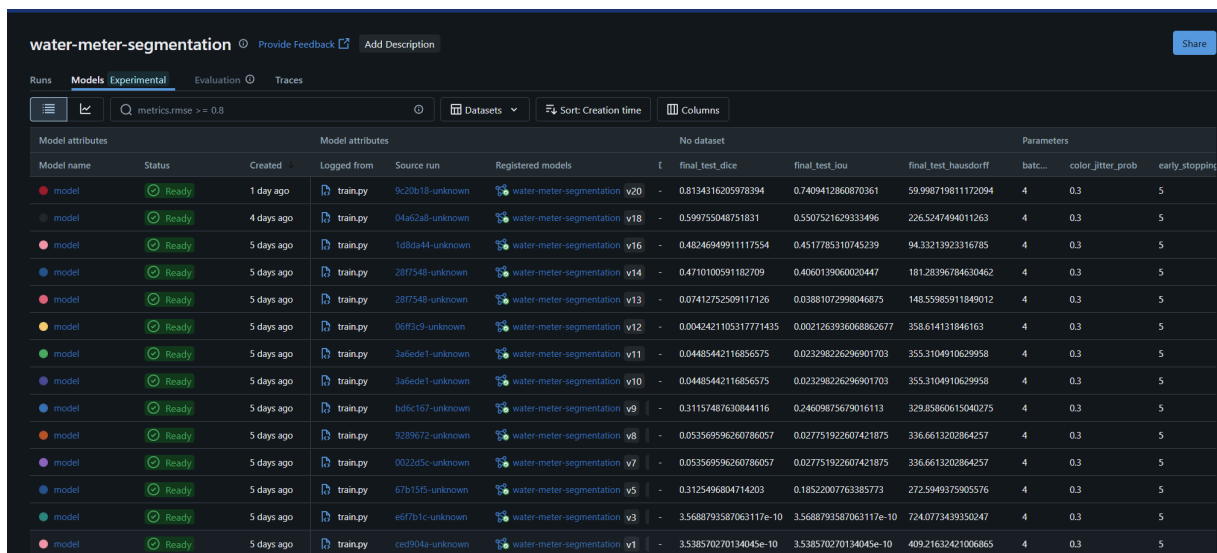
Współczynnik Dice umożliwia ocenę stopnia pokrycia obszaru przewidywanego przez model z obszarem referencyjnym, natomiast IoU mierzy stosunek części wspólnej obu obszarów do ich sumy. Zastosowanie obu metryk pozwala na bardziej kompleksową ocenę jakości predykcji, szczególnie w przypadku niezbalansowanych klas.

Proces uczenia realizowany jest iteracyjnie, z wykorzystaniem zbioru walidacyjnego do monitorowania jakości modelu oraz ograniczania zjawiska przeuczenia. Najlepsza wersja modelu, zgodnie z przyjętymi kryteriami jakości, zapisywana jest jako artefakt i przekazywana do dalszych etapów zautomatyzowanego pipeline'u CI/CD.

4.5 Model jako artefakt w procesie CI/CD

W niniejszej pracy model sztucznej inteligencji traktowany jest jako artefakt podlegający wersjonowaniu, walidacji oraz kontrolowanemu wdrażaniu. Każda wytrenowana wersja modelu rejestrowana jest w systemie MLflow wraz z informacjami o użytym kodzie źródłowym, wersji zbioru danych, konfiguracji hiper-parametrów oraz uzyskanych metrykach jakości. Mechanizm ten umożliwia jednoznaczne powiązanie konkretnego modelu z warunkami jego powstania oraz zapewnia transparentność procesu eksperymentalnego.

Takie podejście zapewnia pełną odtwarzalność wyników oraz tworzy podstawę do porównania podejścia manualnego z procesem zautomatyzowanym w ramach CI/CD.



The screenshot displays the MLflow 'Runs' page for the project 'water-meter-segmentation'. The 'Models' tab is selected, showing a list of 12 model versions. Each row represents a model run with various attributes including name, status, creation time, source, and performance metrics like final_test_dice and final_test_iou.

Model name	Status	Created	Logged from	Source run	Registered models	final_test_dice	final_test_iou	final_test_hausdorff	batch_size	color_litter_prob	early_stopping
model	Ready	1 day ago	train.py	9c20b18-unknown	water-meter-segmentation v20	0.8134316205978394	0.7409412860870361	59.99871981172094	4	0.3	5
model	Ready	4 days ago	train.py	04a62a8-unknown	water-meter-segmentation v18	0.599755048751831	0.5507521629333496	226.5247494011263	4	0.3	5
model	Ready	5 days ago	train.py	1d8de44-unknown	water-meter-segmentation v16	0.4824694991117554	0.4517785310745239	94.33213923316785	4	0.3	5
model	Ready	5 days ago	train.py	287f548-unknown	water-meter-segmentation v14	0.4710100591182709	0.4060139060020447	181.28396784630462	4	0.3	5
model	Ready	5 days ago	train.py	287f548-unknown	water-meter-segmentation v13	0.07412752509117126	0.03881072998046875	148.55985911849012	4	0.3	5
model	Ready	5 days ago	train.py	06f93c9-unknown	water-meter-segmentation v12	0.0042421105317771435	0.002126393608862677	358.614131846163	4	0.3	5
model	Ready	5 days ago	train.py	3a5ede1-unknown	water-meter-segmentation v11	0.04485442116856575	0.023298226296901703	355.3104910629958	4	0.3	5
model	Ready	5 days ago	train.py	3a5ede1-unknown	water-meter-segmentation v10	0.04485442116856575	0.023298226296901703	355.3104910629958	4	0.3	5
model	Ready	5 days ago	train.py	bd5c167-unknown	water-meter-segmentation v9	0.31157487630844116	0.24609875679016113	329.85860615040275	4	0.3	5
model	Ready	5 days ago	train.py	9289672-unknown	water-meter-segmentation v8	0.053569596260786057	0.027751922607421875	336.6613202864257	4	0.3	5
model	Ready	5 days ago	train.py	0022d5c-unknown	water-meter-segmentation v7	0.053569596260786057	0.027751922607421875	336.6613202864257	4	0.3	5
model	Ready	5 days ago	train.py	67b15f5-unknown	water-meter-segmentation v5	0.3125496804714203	0.18522007763385773	272.5949375905576	4	0.3	5
model	Ready	5 days ago	train.py	ed7b1c-unknown	water-meter-segmentation v3	3.5688793587063117e-10	3.5688793587063117e-10	724.0773439350247	4	0.3	5
model	Ready	5 days ago	train.py	ced904a-unknown	water-meter-segmentation v1	3.538570270134045e-10	3.538570270134045e-10	409.21632421006865	4	0.3	5

Rys. 8 Wersjonowanie modeli w rejestrze MLflow

5 Pipeline CI/CD

Rozdział opisuje implementację zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Omówione zostają kolejno: hook Git inicjujący przepływ, struktura workflow GitHub Actions, walidacja i scalanie danych, sekwencja jobów treningowych oraz bramka jakości decydująca o wdrożeniu.

5.1 Hook pre-push

Pierwszym elementem pipeline jest hook Git pre-push [4], instalowany lokalnie na maszynie użytkownika przez skrypt `devops/scripts/install-git-hooks.sh`. Hook uruchamia się automatycznie przed każdym `git push` i sprawdza, czy w zestawie zmian znajdują się pliki w katalogach `WMS/data/training/images/` lub `WMS/data/training/masks/`.

Jeśli zmiany dotyczą danych treningowych, hook:

1. tworzy nową gałąź o nazwie `data/TIMESTAMP` (np. `data/20260215-143022`),
2. commituje do niej wyłącznie nowe pliki z katalogów danych,
3. przekierowuje push na tę gałąź zamiast na `main`.

Całość wykonuje się w mniej niż 10 sekund, ponieważ hook nie wymaga połączenia z AWS ani lokalnych poświadczeń chmurowych. Merge danych z historycznym zbiorem w S3 odbywa się po stronie GitHub Actions.

```
Training data changes detected. Creating data branch...
🔧 Creating branch: data/20260217-013810
🚀 Pushing to data/20260217-013810...
🔍 Checking for training data changes...
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 584 bytes | 584.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'data/20260217-013810' on GitHub by visiting:
remote:   https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization/pull/new/data/20260217-013810
remote:
To https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git
* [new branch]    data/20260217-013810 -> data/20260217-013810
branch 'data/20260217-013810' set up to track 'origin/data/20260217-013810'.

✅ Success! Your changes are now on branch: data/20260217-013810

Next steps:
1. GitHub Actions will validate your data
2. If valid, a Pull Request will be created automatically
3. Training will run automatically
4. If model improves, PR will be auto-approved
5. You (or admin^*) can then merge the PR

error: failed to push some refs to 'https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git'
PS D:\school\bsc\Repositories\Water-Meters-Segmentation-Autimatization> |
```

Rys. 9 Wynik wypchania nowych danych na gałąź `data/TIMESTAMP`

Po zakończeniu działania hooka Git wyświetla komunikat `error: failed to push some refs`. Jest to zachowanie oczekiwane, nie błąd. Hook celowo kończy się kodem niezerowym, aby zablokować oryginalny push do `main`. Dane zostały już pomyślnie wypchnięte na gałąź `data/TIMESTAMP` i GitHub Actions może przejąć kontrolę.

5.2 Struktura workflow GitHub Actions

Główny plik automatyzacji projektu to `.github/workflows/training-data-pipeline.yaml` [6]. Definiuje on trigger, konfigurację współbieżności, uprawnienia i osiem jobów składających się na pipeline.

Trigger

Workflow uruchamia się wyłącznie na push do gałęzi pasujących do wzorca `data/**`. Oznacza to, że kod przesyłany do `main` lub innych gałęzi nie inicjuje treningu. Pipeline startuje tylko wtedy, gdy hook `pre-push` przekieruje dane na gałąź `data/TIMESTAMP`:

```
on:
  push:
    branches:
      - 'data/**'
    workflow_dispatch: {}

concurrency:
  group: training-pipeline
  cancel-in-progress: false

permissions:
  contents: write
  pull-requests: write

jobs:
  merge-and-validate:
    runs-on: ubuntu-latest
    outputs:
      validation_passed: ${ steps.qa.outputs.validation_passed }
      total_images: ${ steps.merge.outputs.total_images }
      new_images: ${ steps.count_new.outputs.new_images }
      existing_images: ${ steps.download.outputs.existing_images }
```

Runnery

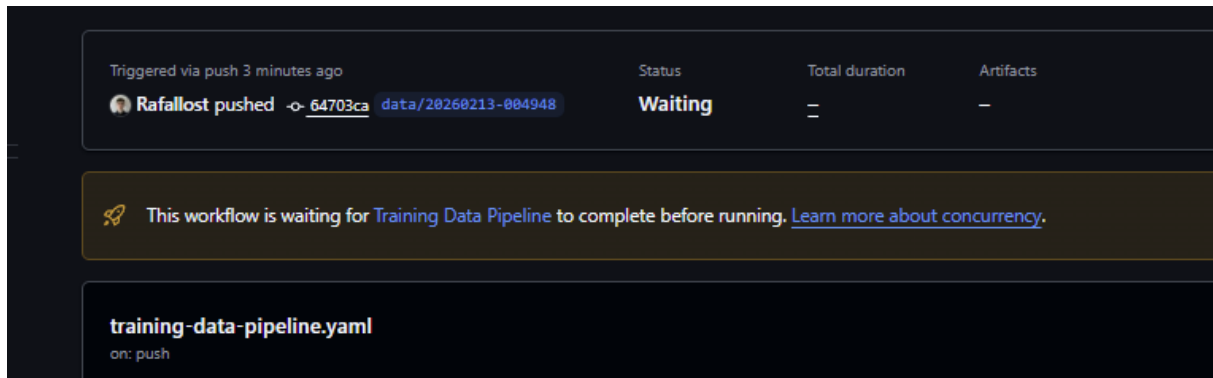
Większość jobów uruchamia się na środowisku zarządzanym przez GitHub (`runs-on: ubuntu-latest`), co nie wymaga utrzymania własnej infrastruktury i nie generuje kosztów podczas oczekiwania. Wyjątkiem jest wyłącznie job `deploy` — wykonuje się na `runs-on: self-hosted`, czyli bezpośrednio na instancji EC2, ponieważ wymaga dostępu do lokalnego klastra k3s i demona Docker.

Komunikacja między jobami

GitHub Actions umożliwia przekazywanie wartości między jobami przez mechanizm *outputs*. Job `merge-and-validate` eksponuje wyniki walidacji (`validation_passed`, liczba obrazów), a job `train` eksponuje flagę `improved`, na podstawie której warunkowane są dalsze joby. Artefakty GitHub Actions służą natomiast do przekazywania samych plików danych (scalony zbiór) między jobem `merge-and-validate` a jobem `train`, który wykonuje się na innym runnerze.

Współbieżność

Klucz `concurrency.cancel-in-progress: false` sprawia, że jeśli użytkownik wypchnął dane kilkakrotnie zanim poprzednie uruchomienie zdążyło się zakończyć, kolejne uruchomienia nie są anulowane tylko trafiają do kolejki i wykonują się po zakończeniu poprzednich. Podejście to chroni przed sytuacją, w której późniejszy push nadpisze wyniki wcześniejszego treningu, zanim zostanie on oceniony przez bramkę jakości.



Rys. 10 Kolejowanie uruchomień workflow w GitHub Actions. Nowe uruchomienie czeka na zakończenie poprzedniego.

5.3 Walidacja danych

Po wypchnięciu gałęzi danych uruchamia się pierwszy job workflow `training-data-pipeline.yaml: merge-and-validate` [6]. Składa się on z dwóch faz.

Faza scalania

Job pobiera istniejące dane z Amazon S3 (za pomocą poświadczeń przechowywanych w sekretach GitHub), scala je z nowo dostarczonymi plikami i aktualizuje manifesty DVC. Pliki o identycznej nazwie i sumie kontrolnej nie są duplikowane. Po zakończeniu scalania zbiór przesyłany jest jako artefakt GitHub Actions do jobu `train`, który wykonuje się na innym runnerze.

Faza walidacji

Skrypt `data-qa.py` weryfikuje scalony zbiór danych pod kątem czterech kryteriów:

- **kompletność par** — dla każdego obrazu w katalogu `images/` musi istnieć maska o tej samej nazwie w `masks/` (porównanie po *stem* nazwy pliku),
- **rozdzielczość** — każdy plik obrazu i maski musi mieć rozdzielczość 512×512 px,
- **binarność masek** — maski mogą zawierać wyłącznie wartości 0 i 255,
- **format pliku** — akceptowane są formaty `.jpg` oraz `.png`.

Wynik walidacji jest publikowany jako komentarz w pull request. W przypadku błędów komentarz zawiera listę plików niezgodnych z wymaganiami, a dalsze joby są pomijane (warunkowanie przez `needs` i `if: needs.merge-and-validate.outputs.validation == 'true'`).

```
python devops/scripts/data-qa.py WMS/data/training/ --output report.json
```

```

Run Data QA Validation

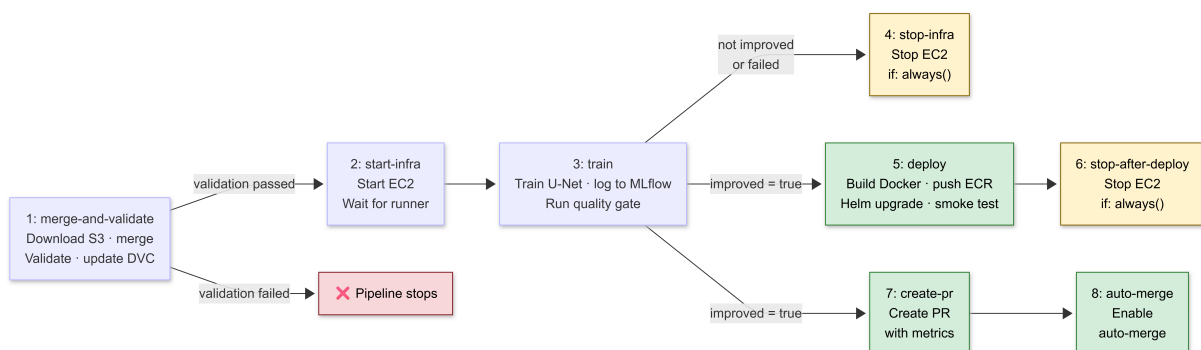
1 ▶ Run echo "Running data quality assurance..."
26 Running data quality assurance...
27 =====
28 ✖ DATA QA FAILED
29 =====
30 Images: 11 | Masks: 11 | Valid pairs: 11
31
32 Errors: 3
33 - id_1_value_13_116: Resolution mismatch
34 - id_2_value_495_341: Resolution mismatch
35 - id_3_value_366_885: Resolution mismatch
36 Error: Process completed with exit code 1.

```

Rys. 11 Przykład komentarza z wynikiem walidacji danych. W tym przypadku wykryto złą rozdzielczość, co skutkuje zatrzymaniem dalszych kroków pipeline.

5.4 Pipeline treningowy

Główny workflow `training-data-pipeline.yaml` składa się z ośmiu jobów. Joby 1–3 wykonywane są sekwencyjnie. Po zakończeniu treningu workflow rozgałęzia się w zależności od wyniku bramki jakości: jeśli model nie poprawił metryk, uruchamia się `stop-infra` (job 4) i pipeline kończy działanie. Jeśli model się poprawił, joby `deploy` (job 5) i `create-pr` (job 7) uruchamiają się równoległe i wzajemnie się nie blokują. Priorytety są celowe: dodanie ulepszanego modelu i scalenie z `main` jest ważniejsze niż wdrożenie, dlatego ewentualna awaria wdrożenia nie blokuje propagacji zatwierdzonego zbioru do repozytorium. Wdrożenie można w każdej chwili powołać przez workflow `release-deploy.yaml` lub `deploy-model.yaml`. Job 6 zatrzymuje EC2 po wdrożeniu, a job 8 finalizuje scalenie pull requesta.



Rys. 12 Sekwencja jobów w workflow `training-data-pipeline.yaml`

Job 1: merge-and-validate

Szczegółowo opisany w podrozdziale 5.3.

Job 2: start-infra

Wykonuje się na hoście GitHub (nie na EC2) i jest odpowiedzialny wyłącznie za uruchomienie infrastruktury. Wywołuje AWS CLI (`aws ec2 start-instances`) z identyfikatorem instancji zapisanym w sekretach repozytorium. Po wysłaniu polecenia co 30 sekund sprawdza API GitHub, czy self-hosted runner przypisany do tej instancji pojawił się w stanie *online* i czeka maksymalnie 10 minut. Dzięki temu każdy kolejny job pewnie trafi na już działającą instancję.

Job 3: train

Job uruchamiany na hoście GitHub (`runs-on: ubuntu-latest`) łączy się z MLflow przez HTTP pod Elastic IP instancji EC2. Pobiera scalony zbiór danych z artefaktu GitHub Actions, uruchamia skrypt `train.py`, który inicjalizuje model U-Net z seedem równym numerowi uruchomienia (`run_number`), trenuje przez zdefiniowaną liczbę epok i loguje metryki `val_dice` oraz `val_iou` do MLflow. Po zakończeniu treningu skrypt `quality-gate.py` pobiera metryki aktualnego modelu Production z rejestru MLflow, porównuje je z wynikami nowego treningu i ustawia output `improved=true` lub `improved=false`, sterując dalszym przebiegiem workflow.

Job 4: stop-infra

Zatrzymuje instancję EC2 gdy trening zakończył się błędem lub model nie poprawił wyników (`if: always() && (train.result != 'success' || improved != 'true')`). Jest to reużywalny workflow wywołujący `aws ec2 stop-instances`. Dzięki warunkowi `always()` job uruchamia się nawet gdy poprzednie joby zakończyły się wyjątkiem, eliminując ryzyko pozostawienia uruchomionej instancji i nieoczekiwanych kosztów.

Job 5: deploy

Uruchamia się wyłącznie jeśli model spełnił kryteria bramki jakości (`if: improved == 'true'`). Wykonuje się na self-hosted runnerze na EC2. Buduje obraz Docker na podstawie pliku `docker/Dockerfile.serve`, taguje go numerem wersji i hashem commita, uwierzytelnia się w Amazon ECR i wgrywa obraz. Następnie wywołuje `helm upgrade`, który zaktualizuje deployment na klastrze k3s do nowej wersji obrazu. Kontener przy starcie pobiera model ze stadium Production z rejestru MLflow, co oddziela wersję obrazu od wersji modelu. Po wdrożeniu wykonywany jest smoke test sprawdzający dostępność endpointów `/health`, `/predict` i `/metrics`.

Job 6: stop-after-deploy

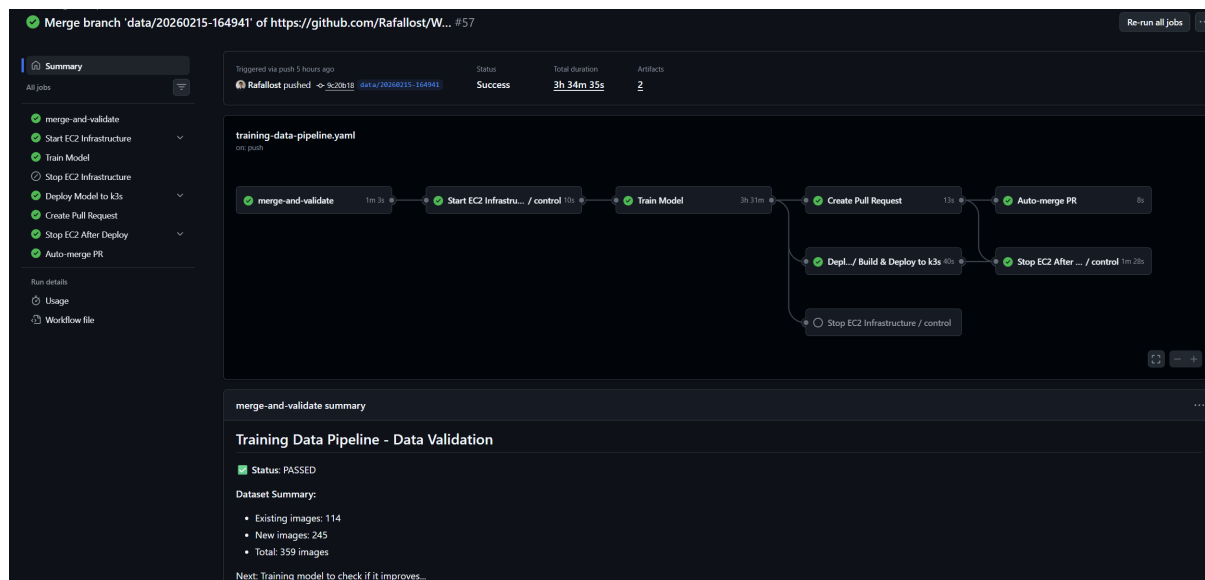
Zatrzymuje instancję EC2 po zakończeniu joba `deploy` (`if: always()`). Stanowi odpowiednik `stop-infra` dla ścieżki, gdy model się poprawił. Istnieje jako osobny job, ponieważ `deploy` korzysta z runnera na EC2. Instancja musi pozostać uruchomiona do końca wdrożenia i może zostać zatrzymana dopiero po jego zakończeniu.

Job 7: create-pr

Uruchamia się równolegle z jobem `deploy`, jeśli model spełnił kryteria bramki jakości. Tworzy pull request z gałęzi danych do `main` z automatycznie generowanym opisem zawierającym zestawienie metryk: wartości `val_dice` i `val_iou` nowego modelu oraz poprzedniego baseline, a także link do uruchomienia MLflow. Pull request scala zaktualizowane manifesty DVC (`images.dvc`, `masks.dvc`) z gałęzią główną, trwale łącząc wersję kodu z wersją danych.

Job 8: auto-merge

Włącza automatyczne scalanie pull requesta przez API GitHub (`gh pr merge -auto`). Scalenie następuje natychmiast po pozytywnym przejściu statusowych sprawdzeń wymaganych przez reguły ochrony gałęzi. W efekcie, jeśli model się poprawił to cały cykl od wypchnięcia danych do scalenia z `main` odbywa się bez żadnej ręcznej interwencji.



Rys. 13 Udany przebieg pipeline CI/CD z automatycznym wdrożeniem nowej wersji modelu

5.5 Bramka jakości

Bramka jakości jest mechanizmem decydującym, czy nowy model jest wystarczająco dobry, żeby trafić do produkcji. Implementacja korzysta z API MLflow Model Registry (opisanego w podrozdziale 2.7) [15]. Skrypt łączy się z MLflow przez `MlflowClient` i pobiera metryki aktualnego modelu Production:

Pobieranie baseline

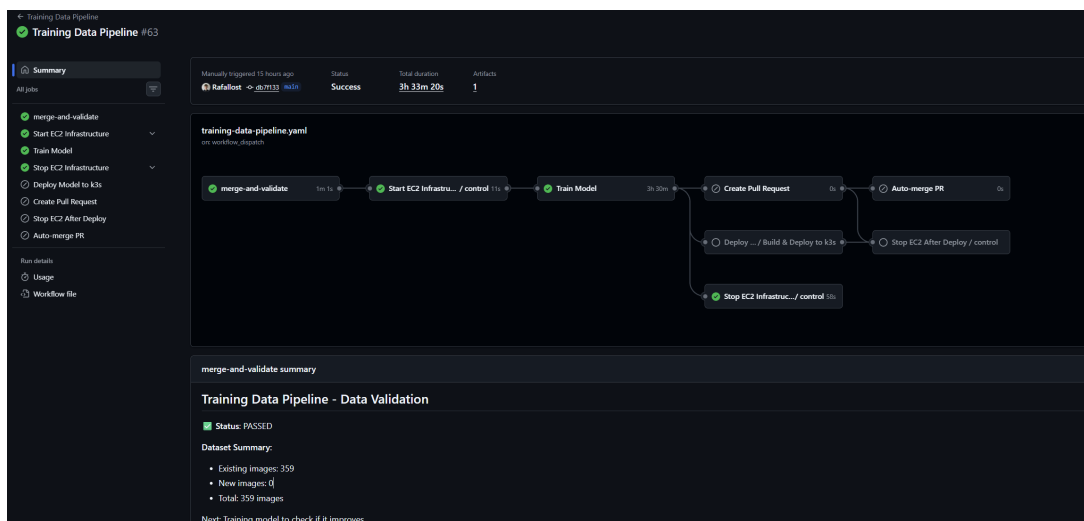
```
client = MlflowClient(tracking_uri=mlflow_uri)
versions = client.get_latest_versions(
    "water-meter-segmentation",
    stages=["Production"]
)
```

Logika decyzyjna

Model jest uznany za ulepszony, jeśli obie metryki ze zbioru testowego — wyznaczone na najlepszym zapisanym modelu po zakończeniu treningu — są wyższe od baseline:

- `final_test_dice > baseline_dice`
- `final_test_iou > baseline_iou`

Jeśli warunek jest spełniony, skrypt promuje nowy model do etapu Production w MLflow i archiwizuje poprzednią wersję. Wynik jest przekazywany jako output do GitHub Actions przez `echo "improved=true">> $GITHUB_OUTPUT`.



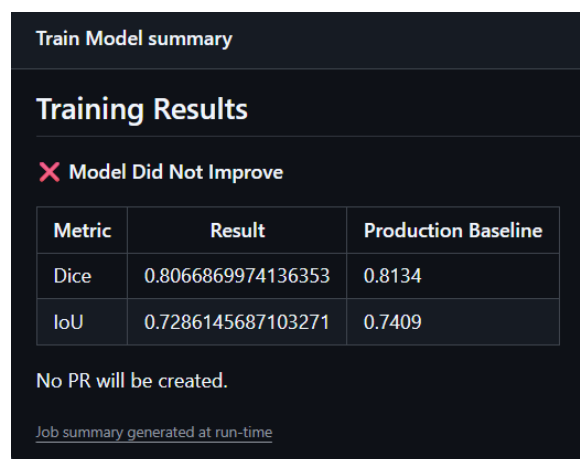
Rys. 14 Przykład z nieudanego przebiegu pipeline CI/CD, który nie spełnia kryteriów bramki jakości

```

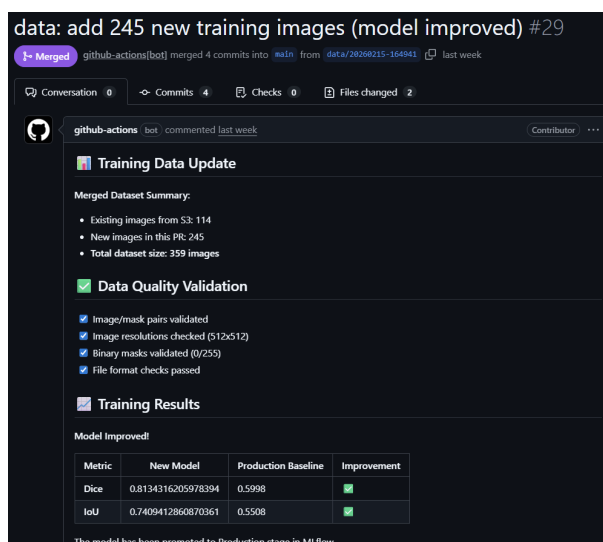
358 =====
359 TRAINING (seed=63)
360 =====
361
362 Results: Dice=0.8067, IoU=0.7286
363 Baseline: Dice=0.8134, IoU=0.7409
364 Improved: ❌ NO
365
366 =====
367 QUALITY GATE RESULTS
368 =====
369 {
370   "run_id": "8a5c55970150428294215e20d7a0db9f",
371   "seed": 63,
372   "metrics": {
373     "dice": 0.8066869974136353,
374     "iou": 0.7286145687103271
375   },
376   "baseline": {
377     "dice": 0.8134316205978394,
378     "iou": 0.7409412860870361
379   },
380   "improved": false
381 }

```

Rys. 15 Wynik bramki jakości modelu, który nie spełnił kryteriów, więc nie został wdrożony



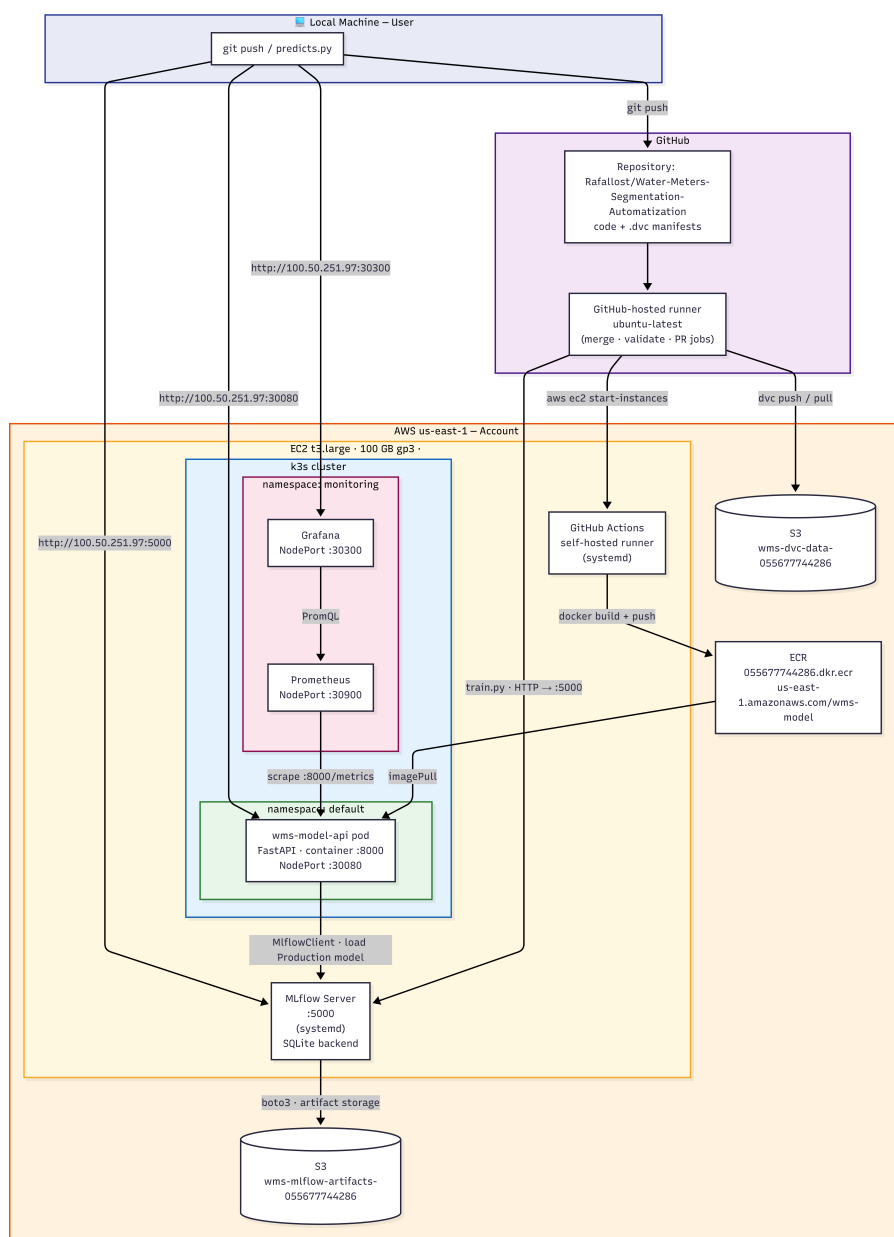
Rys. 16 Podsumowanie metryk modelu, który nie spełnił kryteriów bramki jakości



Rys. 17 Przykład automatycznie scalonego pull requesta z gałęzi danych do main

6 Wdrożenie Systemu

Rozdział opisuje infrastrukturę, na której działa system, sposób jej konfiguracji oraz scenariusze użycia z perspektywy użytkownika.



Rys. 18 Diagram wdrożenia systemu. Szczegółowe zasoby AWS, adresy i porty

Diagram przedstawia topologię wdrożeniową systemu z podziałem na trzy granice: platformę GitHub, usługi AWS oraz środowisko wykonawcze na instancji EC2. Na tej samej maszynie współdzielają dwa procesy `systemd` (runner GitHub Actions i serwer MLflow, opisane w rozdziale 3) oraz klaster `k3s`, w którym uruchomiony jest pod API modelu i stos monitoringu. Konfiguracja `hostNetwork: true` pozwala podowi na dostęp do serwera MLflow przez `localhost:5000`, eliminując potrzebę wystawiania usługi poza klaster.

6.1 Infrastruktura Terraform

Infrastruktura AWS jest zdefiniowana jako kod przy użyciu Terraform [7]. Definicje znajdują się w katalogu `devops/terraform/`. Zastosowana architektura minimalizuje koszty. Używa wyłącznie zasobów niezbędnych do działania projektu i unika kosztownych komponentów zarządzanych przez AWS (EKS, RDS, NAT Gateway, Load Balancer).

Tworzone zasoby:

- **VPC i podsieć publiczna** — prosta sieć z bezpośrednim dostępem do internetu przez Internet Gateway; eliminuje potrzebę NAT Gateway,
- **Security Group** — otwiera porty SSH (22), HTTP (80), NodePort Kubernetes (30000–32767) i port MLflow (5000) dla dozwolonych adresów IP,
- **Instancja EC2** — uruchamiana na żądanie przez workflow [6]; typ instancji konfigurowalny przez zmienną (`t3.large` dla produkcji ML),
- **Amazon S3** — dwa zasobniki: jeden dla artefaktów MLflow, drugi opcjonalnie dla danych treningowych DVC,
- **Amazon ECR** — rejestr obrazów Docker dla API modelu,
- **IAM Role** — rola instancji EC2 z uprawnieniami do odczytu i zapisu S3 oraz ECR.

Infrastruktura uruchamiana jest jednorazowo poleceniem `terraform apply` (rys. 19). Instancja EC2 jest następnie zatrzymywana i uruchamiana automatycznie przez workflow tylko na czas treningu i wdrożenia, co istotnie obniża koszty eksploatacji.

```
module.vpc.aws_route_table_association.public: Creation complete after 1s [id=rtbassoc-0ee193c847ed36b1f]
module.ec2.aws_instance.k3s: Still creating... [00m10s elapsed]
module.ec2.aws_instance.k3s: Creation complete after 15s [id=i-0abe1fe1833dbecde]
module.ec2.aws_instance.k3s: Creation complete after 15s [id=i-0abe1fe1833dbecde]
module.ec2.aws_eip.k3s: Creating...
module.ec2.aws_eip.k3s: Creation complete after 4s [id=eipalloc-07c6c1e01190ef221]
module.ec2.aws_eip.k3s: Creation complete after 4s [id=eipalloc-07c6c1e01190ef221]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

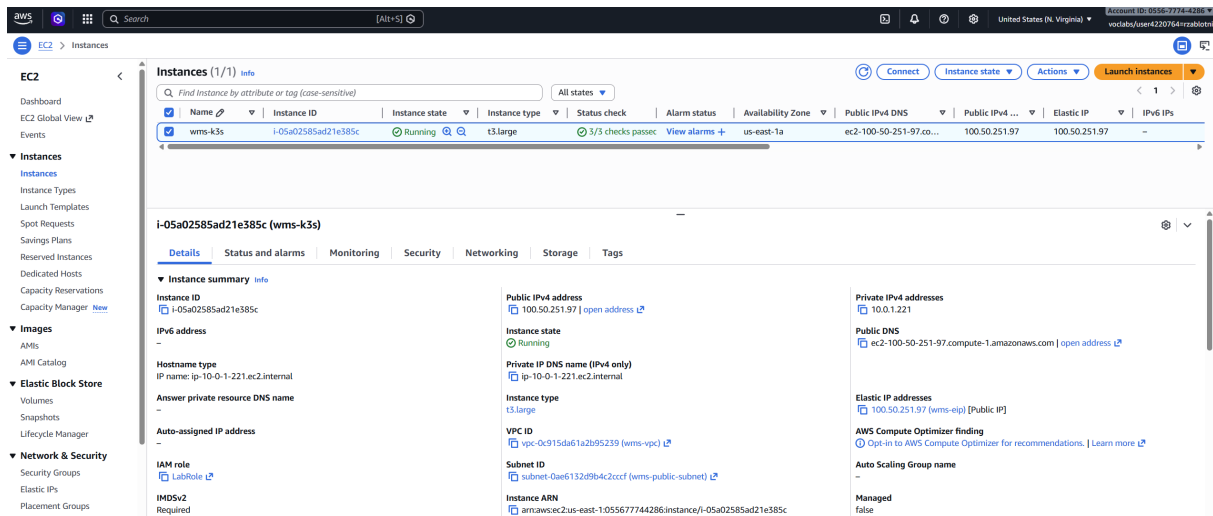
dvc_bucket = "wms-dvc-data-055677744286"
ec2_instance_id = "i-0abe1fe1833dbecde"
ec2_public_ip = "34.199.41.233"
ecr_repository_url = "055677744286.dkr.ecr.us-east-1.amazonaws.com/wms-model"
grafana_port_forward = "kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3000:80"
mlflow_bucket = "wms-mlflow-artifacts-055677744286"
mlflow_url = "http://34.199.41.233:5000"
monitoring_enabled = true
prometheus_port_forward = "kubectl port-forward -n monitoring svc/kube-prometheus-stack-prometheus 9090:9090"
ssh_command = "ssh -i ~/.ssh/vockey.pem ec2-user@34.199.41.233"
ssh_tunnel_grafana = "ssh -i ~/.ssh/vockey.pem -L 3000:localhost:3000 ec2-user@34.199.41.233"
```

Rys. 19 Wynik polecenia `terraform apply` — lista 12 utworzonych zasobów AWS

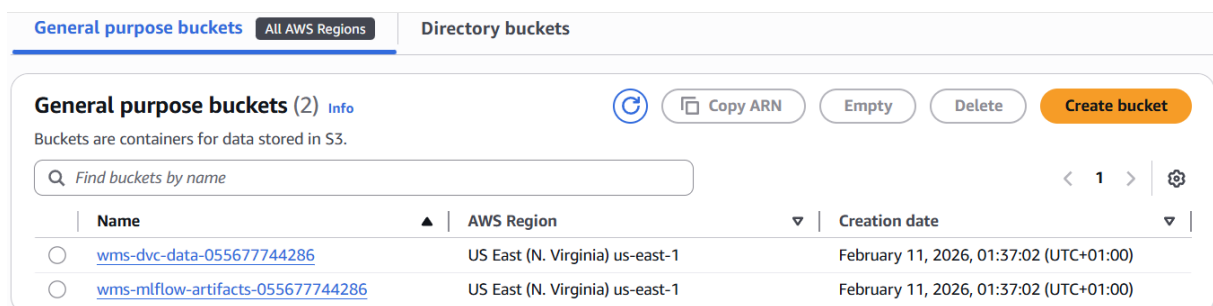
Kod Terraform zorganizowany jest w moduły wywoływane z `main.tf`:

- `modules/vpc/` — sieć VPC, podsieć publiczna, Internet Gateway,
- `modules/s3-mlops/` — zasobniki S3 i uprawnienia,
- `modules/ecr/` — rejestr obrazów Docker,
- `modules/ec2-k3s/` — instancja EC2, Elastic IP, skrypt `user-data`,
- `variables.tf` — deklaracje zmiennych z wartościami domyślnymi,
- `terraform.tfvars` — konkretne wartości zmiennych (np. nazwy zasobników S3).

Rysunki 20 i 21 przedstawiają odpowiednio instancję EC2 i zasobniki S3 w AWS.



Rys. 20 Instancja EC2 wms-k3s w konsoli AWS



Rys. 21 Zasobniki S3: wms-dvc-data-* dla danych treningowych i wms-mlflow-artifacts-* dla artektów

6.2 Konfiguracja EC2 i k3s

Konfiguracja instancji EC2 odbywa się automatycznie przy pierwszym uruchomieniu za pomocą skryptu `user-data`. Skrypt jest szablonowany przez Terraform, który przy generowaniu pliku podstawiane są zmienne takie jak adres zasobnika S3 i region AWS, dzięki czemu nie ma żadnych wartości zakodowanych na stałe.

Kroki wykonywane przez `user-data`:

1. Instalacja zależności systemowych: `dnf install -y git curl libicu`.
2. Instalacja k3s — uproszczonego Kubernetes [17].
3. Instalacja MLflow i boto3: `pip3 install -ignore-installed mlflow boto3`.
4. Konfiguracja MLflow jako usługi systemd:

```
[Service]
ExecStart=mlflow server \
  --backend-store-uri sqlite:///opt/mlflow/mlflow.db \
  --default-artifact-root s3://${mlflow_bucket}/ \
  --host 0.0.0.0
```

5. Instalacja GitHub Actions runnera jako usługi systemd.

Instancja korzysta z Amazon Linux 2023, który oferuje nowszy OpenSSL 3.0 i wsparcie producenta do 2028 roku. Po zakończeniu user-data instancja jest gotowa do przyjęcia zadań z GitHub Actions bez dalszej konfiguracji ręcznej.

Architektura k3s. K3s (opisany w podrozdziale 2.9) działa na instancji t3.large (8 GB RAM) ze znaczącym zapasem zasobów dla poda z modelem; pełne Kubernetes samo w sobie wymagałoby podobnej ilości RAM.

W projekcie użyty jest klaster jednorazowy (single-node): jeden węzeł pełni jednocześnie rolę control plane i workera. Taka architektura jest wystarczająca dla jednej działającej aplikacji i eliminuje koszty dodatkowych instancji EC2. Wysoka dostępność (HA) nie była wymaganiem. System jest przeznaczony do demonstracji i testowania, nie do środowiska produkcyjnego.

Strategia aktualizacji. Dla Deploymentu zastosowano domyślną strategię RollingUpdate: k3s najpierw uruchamia nowy pod, a dopiero po jego gotowości usuwa stary. Oznacza to zerowy przestój (zero-downtime deployment) przy aktualizacji modelu. Działa to jednak tylko wtedy, gdy instancja EC2 jest uruchomiona. W przeciwnym wypadku klaster jest niedostępny.

Rollbacki i wersjonowanie. Rollbacki na poziomie Kubernetes nie są rozbudowane celowo. Mechanizmem wersjonowania modeli jest MLflow, gdzie każda wersja modelu jest zarejestrowana w rejestrze MLflow ze swoimi metrykami. Rollback polega na przestawieniu stadium modelu w MLflow z Production na poprzednią wersję i ponownym uruchomieniu deploy-model.yaml. Helm zachowuje historię ostatnich wdrożeń (helm history wms-model), co umożliwia też cofnięcie konfiguracji Kubernetes bez ponownego treningu.

Rysunek 22 potwierdza poprawne działanie usług systemd, a rysunek 23 przedstawia aktualny stan podów klastra k3s.

```
ec2-user@ip-10-0-1-221:~$ systemctl status mlflow
● mlflow.service - MLflow Tracking Server
   Loaded: loaded (/etc/systemd/system/mlflow.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-02-18 00:48:51 UTC; 1h 27min ago
     Main PID: 2063 (mlflow)
       Tasks: 9 (limit: 9297)
      Memory: 481.4M
         CPU: 9.265s
    CGroup: /system.slice/mlflow.service
            └─2063 /usr/bin/python3 /usr/local/bin/mlflow server --backend-store-uri sqlite:///opt/mlflow/mlflow.db --default-artifact-root s3://aws-mlflow-artifacts-05567744286/ --host 0.0.0.0 --workers 2 --gunicorn-opts "--timeout 300 --keep-alive 120"
            └─2069 /usr/bin/python3 -m gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app
            └─2410 /usr/bin/python3 -m gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app
            └─2411 /usr/bin/python3 -m gunicorn --timeout 300 --keep-alive 120 -b 0.0.0.0:5000 -w 2 mlflow.server:app

Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:51:59 ip-10-0-1-221.ec2.internal mlflow[2410]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: 2026/02/18 00:52:20 INFO mlflow.store.db.utils: Creating initial mlflow database tables...
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: 2026/02/18 00:52:20 INFO mlflow.store.db.utils: Updating database tables...
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Context impl SQLiteImpl.
Feb 18 00:52:20 ip-10-0-1-221.ec2.internal mlflow[2411]: INFO [alembic.runtime.migration] Will assume non-transactional DDL.
ec2-user@ip-10-0-1-221:~$ systemctl status actions-runner
● actions-runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service - GitHub Actions Runner (Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221)
   Loaded: loaded (/etc/systemd/system/actions.runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-02-18 00:48:51 UTC; 1h 28min ago
     Main PID: 2054 (runsvch.sh)
       Tasks: 28 (limit: 9297)
      Memory: 153.4M
         CPU: 2.878s
    CGroup: /system.slice/actions.runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service
            └─2054 /bin/bash /home/ec2-user/actions.runner/runsvch.sh
            └─2060 /usr/bin/node /home/ec2-user/actions.runner/bin/RunnerService.js
            └─2127 /home/ec2-user/actions.runner/bin/Runner.Listener run --startuptime service

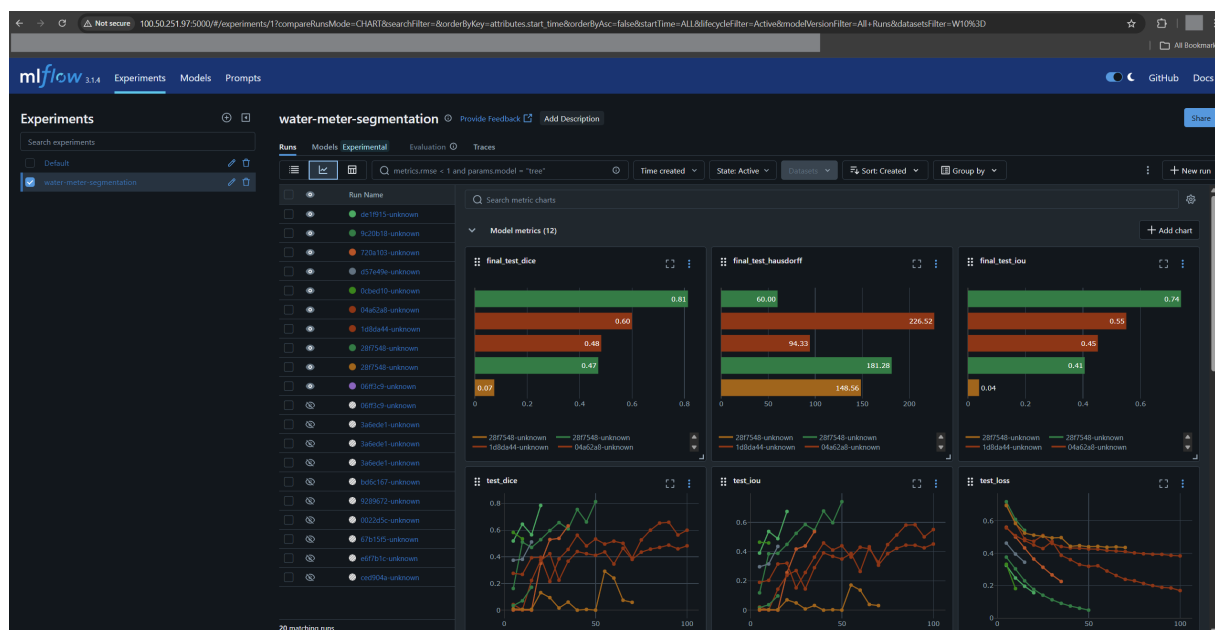
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal systemd[1]: Started actions.runner.Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221.service - GitHub Actions Runner (Rafalost-Water-Meters-Segmentation-Autimatization.ip-10-0-1-221).
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsvch.sh[2054]: /path=/home/ec2-user/.local/bin:/home/ec2-user/bin:/usr/local/bin:/usr/bin:/usr/sbin
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: Starting Runner.Listener with startup type: service
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: Started listener process, pid: 2127
Feb 18 00:48:51 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: Started running service
Feb 18 00:48:54 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: ✓ Connected to GitHub
Feb 18 00:48:54 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: Current runner version: '2.331.0'
Feb 18 00:48:54 ip-10-0-1-221.ec2.internal runsvch.sh[2060]: 2026-02-18 00:48:54Z: Listening for jobs
ec2-user@ip-10-0-1-221:~$
```

Rys. 22 Stan usług mlflow i actions.runner w systemd — obie usługi aktywne

```
[ec2-user@ip-10-0-1-221 ~]$ kubectl get pods -A
NAMESPACE      NAME                                     READY   STATUS    RESTARTS   AGE
default         wms-model-m1-model-d74dcb96f-wtn8z    1/1     Running   2 (89m ago)  97m
kube-system     coredns-67d69c9b5b-xrvzt              1/1     Running   39 (94m ago)  5d6h
kube-system     helm-install-traefik-crd-lfp4z         0/1     Completed 0           5d6h
kube-system     helm-install-traefik-mljg7             0/1     Completed 2           5d6h
kube-system     local-path-provisioner-546dfc6456-x6m4t 1/1     Running   40 (94m ago)  5d6h
kube-system     metrics-server-7b9c9c4b9c-52nmg        1/1     Running   39 (94m ago)  5d6h
kube-system     svclb-traefik-3f01b321-p2jcr           2/2     Running   78 (94m ago)  5d6h
kube-system     traefik-55db578d67-p44ld              1/1     Running   39 (94m ago)  5d6h
monitoring      alertmanager-kube-prometheus-stack-alertmanager-0 2/2     Running   16 (94m ago)  45h
monitoring      kube-prometheus-stack-grafana-65c4485b4d-wrqjs 3/3     Running   6 (94m ago)  97m
monitoring      kube-prometheus-stack-kube-state-metrics-8f4c577fd-7bx2d 1/1     Running   8 (94m ago)  45h
monitoring      kube-prometheus-stack-operator-5bd989bd85-49z16 1/1     Running   2 (94m ago)  97m
monitoring      kube-prometheus-stack-prometheus-node-exporter-rckwh 1/1     Running   8 (94m ago)  45h
monitoring      prometheus-kube-prometheus-stack-prometheus-0 2/2     Running   16 (94m ago)  45h
```

Rys. 23 Pody klastra k3s — `kubectl get pods -A`. Widoczny pod API modelu oraz stos monitorowania.

Interfejs MLflow dostępny jest pod adresem `http://100.50.251.97:5000` (rys. 24).



Rys. 24 Interfejs MLflow (`http://100.50.251.97:5000`) — lista eksperymentów i przebiegów treningowych z metrykami

6.3 Dostępne porty i usługi

Security Group Terraform otwiera wyłącznie porty niezbędne do działania systemu. Tabela 2 zestawia wszystkie dostępne punkty dostępu.

Tabela 2 Porty dostępne na instancji EC2 `wms-k3s`

Port	Usługa	Przeznaczenie
22	SSH	Zarządzanie instancją EC2 (tylko IP operatora)
5000	MLflow	Interfejs śledzenia eksperymentów i rejestr modeli
30080	API modelu (NodePort)	Predykcja segmentacji — endpoint <code>/predict</code>
30300	Grafana (NodePort)	Dashboard wizualizacji metryk
30900	Prometheus (NodePort)	Interfejs zapytań i podgląd <i>Targets</i>

Ruch przychodzący na porty 5000, 30080, 30300 i 30900 jest dozwolony z dowolnego adresu IP (0.0.0.0/0), ponieważ EC2 nie jest wyposażone w Load Balancer z certyfikatem TLS, a połączenia odbywają się po HTTP. Port SSH (22) jest ograniczony do konkretnego adresu IP operatora skonfigurowanego w `terraform.tfvars`. Konfigurację Security Group przedstawia rysunek 25.

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sgr-0f544380b4c910e65	IPv4	Custom TCP	TCP	30900	46.112.114.36/32	Prometheus
-	sgr-0201321fd5a004a2f	IPv4	Custom TCP	TCP	30080	0.0.0.0/0	The application
-	sgr-099bda2b9f4c6cf71	IPv4	SSH	TCP	22	46.112.114.36/32	SSH from my IP (auto-d...
-	sgr-00c3fe3405d3170b3	IPv4	Custom TCP	TCP	30300	46.112.114.36/32	Grafana
-	sgr-00847d17feffc5aa	IPv4	Custom TCP	TCP	8000	0.0.0.0/0	HTTP from my IP (auto...
-	sgr-05738bf1210f436ee	IPv4	Custom TCP	TCP	5000	0.0.0.0/0	MLflow from anywhere...

Rys. 25 Security Group instancji EC2 — reguły ruchu przychodzącego z otwartymi portami usług

6.4 Konfiguracja Helm

W projekcie Helm [8] wykorzystywany jest dwójako: do wdrożenia własnego charta API modelu oraz do konfiguracji zewnętrznego stosu monitorowania.

Chart API modelu. Katalog `devops/helm/ml-model/` zawiera własny chart Kubernetes dla serwera API. Zamiast zarządzać oddzielnymi plikami manifestów, całą aplikację opisuje jeden zestaw szablonów YAML sparametryzowanych zmiennymi. Bez Helm każda aktualizacja wymagałaby ręcznej edycji kilku plików i kolejnych wywołań `kubectl apply`. Helm sprowadza to do jednej komendy:

```
helm upgrade --install wms-model devops/helm/ml-model/ \
  -f infrastructure/helm-values.yaml \
  --namespace default --create-namespace
```

Komenda ta działa zarówno przy pierwszym wdrożeniu (*install*) jak i przy aktualizacji (*upgrade*). Helm sam sprawdza czy release już istnieje. Helm zachowuje historię wdrożeń (`helm history wms-model`), co umożliwia szybki rollback do poprzedniej wersji.

Struktura charta:

```
devops/helm/ml-model/
+-- Chart.yaml          <- metadane (nazwa, wersja charta)
+-- values.yaml         <- wartosci domyslne (NodePort 30080, limity
    RAM/CPU)
\-- templates/
    +-- _helpers.tpl    <- funkcje pomocnicze Helm
    +-- deployment.yaml <- zasob Deployment
    +-- service.yaml    <- zasob Service (NodePort 30080)
    \-- servicemonitor.yaml <- ServiceMonitor dla Prometheus
```

Szablony korzystają ze składni Go Templates i odwołują się do wartości przez `.Values.*`. Plik `infrastructure/helm-values.yaml` nadpisuje wartości domyślne charta dla konkretnego wdrożenia.

Najważniejsze fragmenty:

```
image:
  repository: "055677744286.dkr.ecr.us-east-1.amazonaws.com/wms-model"
  pullPolicy: Always
  tag: "latest"

imagePullSecrets:
  - name: ecr-secret      # dane do logowania do ECR

env:
  - name: MLFLOW_TRACKING_URI
    value: "http://localhost:5000"  # MLflow na hoscie EC2 (hostNetwork)
  - name: MODEL_VERSION
    value: "production"

resources:
  limits:
    memory: "1Gi"
    cpu: "500m"
  requests:
    memory: "512Mi"
    cpu: "100m"
```

Warto zwrócić uwagę na `MLFLOW_TRACKING_URI: localhost:5000`; kontener korzysta z `hostNetwork: true`, co pozwala mu odwoływać się do MLflow działającego bezpośrednio na maszynie EC2, bez konieczności dodatkowej usługi Kubernetes. Przy każdym wdrożeniu GitHub Actions podaje aktualny hash commita jako `-set image.tag=${COMMIT_SHA}`, co zapewnia jednoznaczność identyfikacji wersji obrazu.

Postęp wdrożenia API można śledzić w czasie rzeczywistym poleceniem:

```
kubectl rollout status deployment/wms-model-ml-model --watch
```

Po wdrożeniu API jest dostępne pod adresem `http://100.50.251.97:30080`.

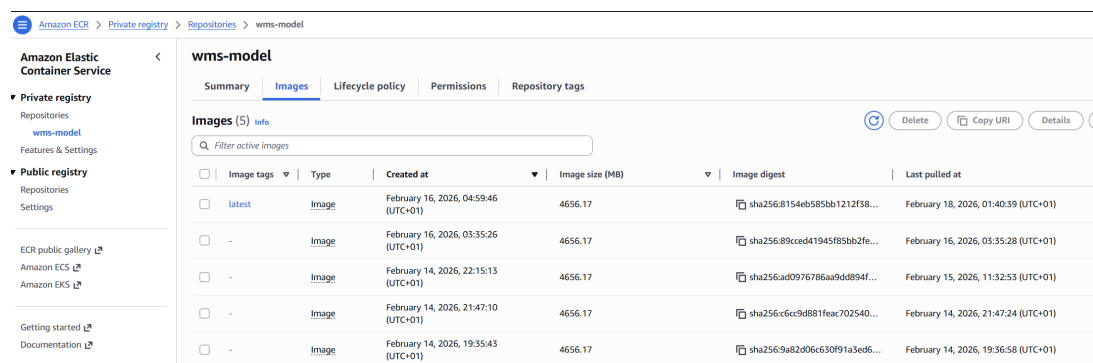


Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
latest	Image	February 16, 2026, 04:59:46 (UTC+01)	4656.17	sha256:8154eb585bb1212f38...	February 18, 2026, 01:40:39 (UTC+01)
-	Image	February 16, 2026, 03:35:26 (UTC+01)	4656.17	sha256:89cccd41945f85bb2fe...	February 16, 2026, 03:35:28 (UTC+01)
-	Image	February 14, 2026, 22:15:13 (UTC+01)	4656.17	sha256:ad0976786aa9dd8894f...	February 15, 2026, 11:32:53 (UTC+01)
-	Image	February 14, 2026, 21:47:10 (UTC+01)	4656.17	sha256:6cc9d881feac702540...	February 14, 2026, 21:47:24 (UTC+01)
-	Image	February 14, 2026, 19:35:43 (UTC+01)	4656.17	sha256:9a82d06c630f91a3ed6...	February 14, 2026, 19:36:58 (UTC+01)

Rys. 26 Repozytorium `wms-model` w Amazon ECR z obrazami oznaczonymi hashem commita

Dostępne endpointy API to `/health` (status usługi), `/predict` (predykcja segmentacji) i `/metrics` (metryki Prometheus). Odpowiedź endpointu `/health` przedstawia rysunek 27a, a pełną listę eksponowanych metryk prezentuje rysunek 27b.


```
← → ↻ ⚠ Not secure 100.50.251.97:30080/health

Pretty print ☐

{"status":"healthy","model_loaded":true,"device":"cpu"}
```

(a) Odpowiedź endpointu /health wdrożonego API modelu

```
# HELP wms_predict_latency_seconds_created Prediction latency in seconds
# TYPE wms_predict_latency_seconds_created gauge
wms_predict_latency_seconds_created 1.7713757733752115e+09
# HELP wms_predict_errors_total Total number of prediction errors
# TYPE wms_predict_errors_total counter
wms_predict_errors_total 0.0
# HELP wms_predict_errors_created Total number of prediction errors
# TYPE wms_predict_errors_created gauge
wms_predict_errors_created 1.7713757733752952e+09
# HELP wms_model_loaded Model loaded status (1=loaded, 0=not loaded)
# TYPE wms_model_loaded gauge
wms_model_loaded 1.0
```

(b) Metryki Prometheus eksponowane pod adresem /metrics: liczba predykcji, czasy odpowiedzi i błędy

Rys. 27 Endpointy diagnostyczne API modelu

6.5 Stos monitorowania

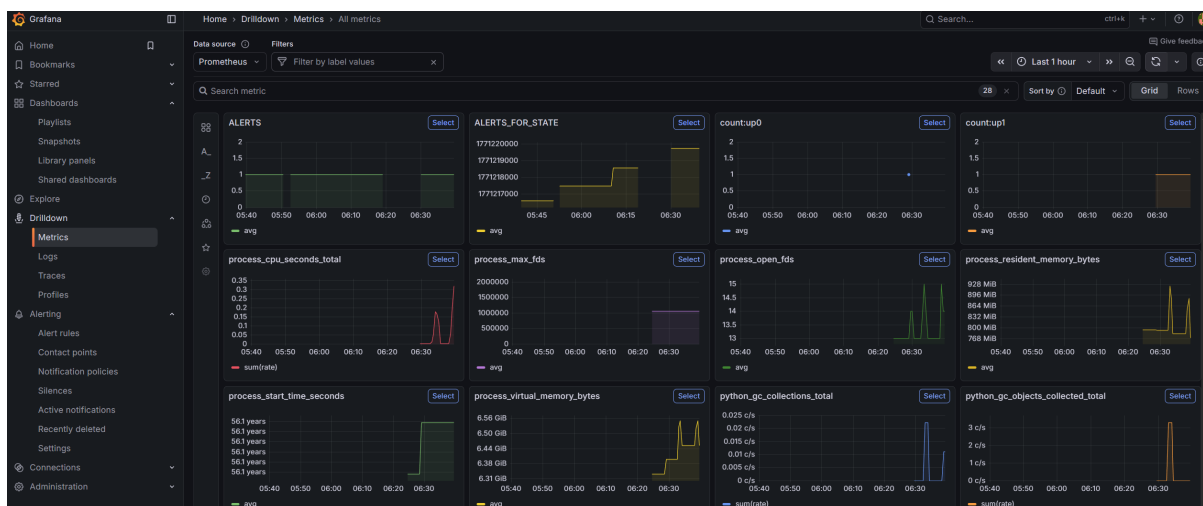
Monitoring oparty na Prometheus [16] i Grafana wdrażany jest jako oddzielny stos na tym samym klastrze k3s. Składa się z trzech komponentów:

- **Prometheus** — zbiera metryki z endpointu /metrics API modelu co 15 sekund; konfiguracja scraping w `prometheus.yml`,
- **Grafana** — wizualizuje metryki na predefiniowanych dashboardach; dostępna na porcie NodePort 30300,
- **kube-state-metrics** — dostarcza metryki Kubernetes (stan podów, zużycie zasobów węzła).

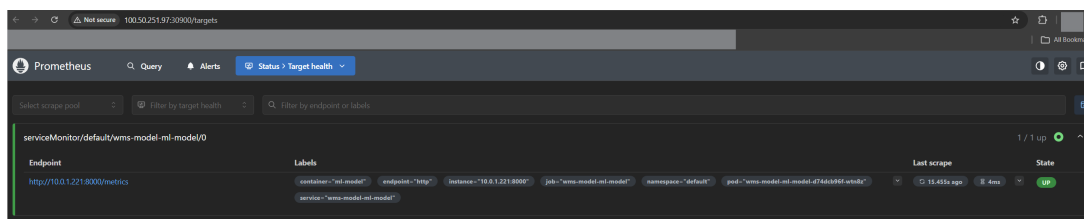
API modelu eksponuje metryki w formacie Prometheus przy użyciu biblioteki `prometheus_client`. Śledzone metryki to:

- `wms_predictions_total` — łączna liczba żądań predykcji (Counter),
- `wms_predict_latency_seconds` — histogram czasów odpowiedzi,
- `wms_predict_errors_total` — liczba błędów (Counter),
- `wms_model_loaded` — czy model jest załadowany (Gauge, wartość 0 lub 1).

Działanie stosu monitorowania ilustruje dashboard Grafana (rys. 28) oraz widok Targets w Prometheus (rys. 29).



Rys. 28 Dashboard Grafana (<http://100.50.251.97:30300>) — metryki działającego API modelu



Rys. 29 Strona *Targets* w Prometheus (:30900) — endpoint `/metrics` API w stanie UP

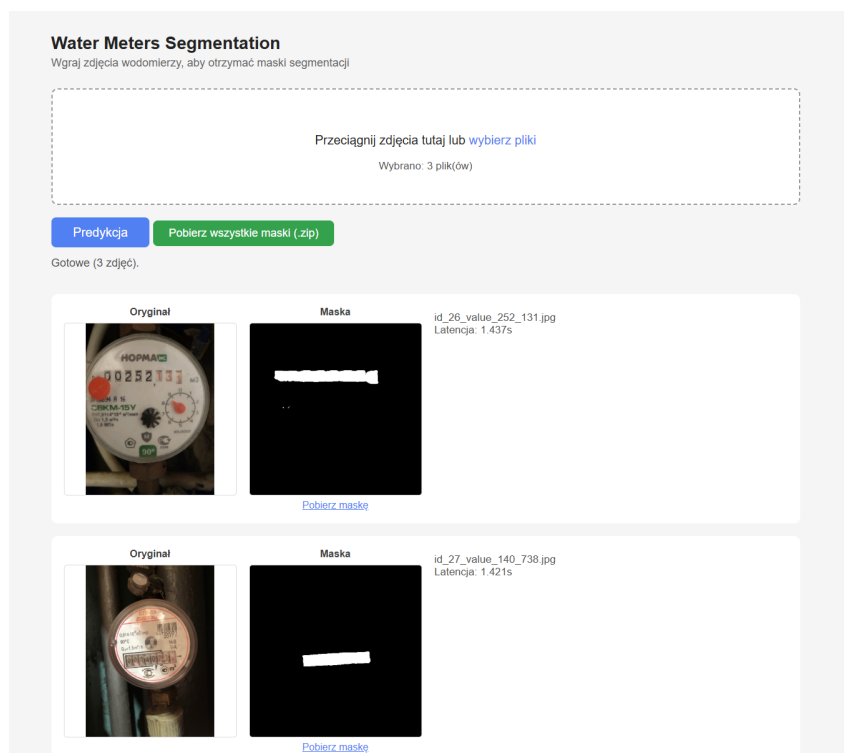
6.6 Workflow użytkownika

6.6.1 Scenariusz 1: Ręczne wdrożenie i korzystanie z API (zalecany)

Jest to zalecany sposób korzystania z systemu dla operatora chcącego uruchomić lub zaktualizować działające API pod adresem `http://100.50.251.97:30080/`. Cały proces odbywa się przez interfejs GitHub Actions i nie wymaga żadnych lokalnych poświadczeń AWS.

Krok 1: Wdrożenie modelu. W zakładce *Actions* należy uruchomić workflow *Release & Deploy* (`release-deploy.yaml`) klikając *Run workflow*. Workflow automatycznie uruchamia instancję EC2, buduje obraz Docker, wypycha go do ECR, pobiera najnowszy model ze stadium Production z MLflow, aktualizuje klaster k3s przez Helm, a po weryfikacji zatrzymuje EC2.

Krok 2: Uruchomienie instancji do predykcji. Po wdrożeniu model jest zainstalowany na klastrze, ale EC2 jest zatrzymany w celu oszczędności. Aby wysyłać zapytania do API, należy uruchomić instancję workflow *Manual EC2 Control* z akcją *start*. Workflow wyświetla aktualne IP i URL MLflow w podsumowaniu uruchomienia.



Rys. 30 Działający interfejs webowy API modelu: segmentacja trzech zdjęć wodomierzy z maskami

Po zakończeniu pracy należy ponownie uruchomić *Manual EC2 Control* z akcją *stop*, aby nie generować zbędnych kosztów.

6.6.2 Scenariusz 2: Wdrożenie przez automatyczny pipeline

Jak opisano w rozdziale 5, dostarczenie nowych danych treningowych przez `git push` wyzwała pełny pipeline. Jeśli wytrenowany model przeszedł bramkę jakości, pipeline automatycznie wdraża go na klaster k3s przez `deploy-model.yaml`, a następnie zatrzymuje instancję EC2. Po zakończeniu pipeline'u aplikacja jest zainstalowana na klastrze i gotowa do działania. Aby skorzystać z API, wystarczy uruchomić instancję EC2 za pomocą workflow *Manual EC2 Control* (zakładka *Actions* w GitHub, akcja *start*). Po uruchomieniu workflow wyświetla adres IP w podsumowaniu, a endpoint `/predict` jest natychmiast dostępny.

Należy zaznaczyć, że wdrożenie następuje tylko gdy model poprawił metryki. Jeśli trening nie przeszedł bramki jakości, klaster k3s nadal serwuje poprzednią wersję. Do kontrolowanego wdrożenia niezależnego od treningu służy Scenariusz 1.

6.7 Dostępne operacje zarządzania wdrożeniem

System udostępnia cztery workflow GitHub Actions do zarządzania infrastrukturą i wdrożeniami. Tabela 3 zestawia je wraz z wyzwalaczami i przeznaczeniem.

Tabela 3 Workflow GitHub Actions dostępne w systemie

Workflow	Wyzwalacz	Przeznaczenie
ci	Push do main/develop, PR do main	Lintowanie kodu (flake8), sprawdzanie formatowania (black) i uruchamianie testów jednostkowych (pytest) z raportem pokrycia kodu. Nie wymaga infrastruktury AWS.
release-deploy	Push do main (zmiany kodu/Docker/Helm), workflow_dispatch	Wdrożenie modelu na klaster k3s bez treningu: build Docker, push do ECR, aktualizacja Helm. Używany przy zmianach kodu serwowania lub jako ręczne ponowne wdrożenie. EC2 uruchamiany i zatrzymywany automatycznie.
ec2-manual-control	Tylko workflow_dispatch	Ręczne uruchomienie lub zatrzymanie instancji EC2. Po starcie wyświetla aktualne IP i URL MLflow w podsumowaniu. Używany gdy potrzebny jest dostęp do API lub interfejsu MLflow.
deploy-model	Reużywalny (workflow_call), workflow_dispatch	Właściwy krok wdrożenia wywołany przez training-data-pipeline i release-deploy. Można też uruchomić samodzielnie. Działa na self-hosted runnerze EC2, uwierzytelniając się przez rolę IAM instancji.

Żaden z powyższych workflow nie wymaga lokalnych poświadczeń AWS na komputerze operatora. Workflow uruchamiane na runnerze `ubuntu-latest` (start/stop EC2) korzystają z sekretów repozytorium GitHub, natomiast `deploy-model` działa na self-hosted runnerze EC2 i uwierzytelnia się przez rolę IAM przypisaną do instancji.

7 Analiza Wyników i Działania Systemu

7.1 Cel analizy i kryteria oceny

Rozdział ma charakter porównawczy i zestawia ze sobą dwa podejścia do rozwijania modelu AI: manualne oraz zautomatyzowane CI/CD.

Aby porównanie było jednoznaczne, analizę przeprowadzono w sześciu wymiarach:

- nakład pracy człowieka i całkowity czas realizacji cyklu
- jakość modelu (Dice, IoU) i skuteczność bramki jakości,
- koszty infrastruktury,
- zużycie zasobów,
- ryzyko błędu ludzkiego
- skalowalność procesu.

7.2 Metodologia porównania

Porównano dwa scenariusze realizujące ten sam cel biznesowy (dodanie nowych danych i dostarczenie działającego modelu):

1. **Podejście ręczne** — inżynier ręcznie wykonuje kolejne kroki: walidację danych, trening, ocenę metryk, decyzję o promocji modelu i wdrożenie.
2. **Podejście zautomatyzowane (zrealizowany pipeline)** — operator dostarcza dane przez `git push`, a dalszy przebieg realizują skrypty i workflow.

Utrzymanie porównywalnego środowiska narzędziowego w obu podejściach było kluczowe. Pozwala przypisać różnice w nakładzie pracy i ryzyku błędu automatyzacji procesu, a nie zmianie technologii. Dokładniejsze wskazanie granicy między podejściami ułatwia poniższe zestawienie:

- **Narzędzia wspólne dla obu podejść:** EC2 (środowisko obliczeniowe), MLflow (śledzenie eksperymentów, metryk i wersji modeli), k3s z Helm (wdrożenie), Docker (konteneryzacja API).
- **Narzędzia wyłącznie dla podejścia zautomatyzowanego:** GitHub Actions (orkiestracja pipeline), DVC (wersjonowanie *danych treningowych* i synchronizacja z S3), hook `pre-push` Git (automatyczne tworzenie gałęzi danych), Terraform (infrastruktura jako kod).

Porównanie obejmuje sześć wymiarów:

- **Nakład pracy człowieka i czas całkowity** (podsekcja 7.3),
- **Jakość modelu** (podsekcja 7.4),
- **Koszty infrastruktury i obsługi** (podsekcja 7.5),
- **Zużycie zasobów** (podsekcja 7.6),
- **Ryzyko błędu ludzkiego** (podsekcje 7.7.2).
- **skalowalność procesu** (podsekcje 7.7.3).

Istotne ograniczenie metodologiczne: czasy dla podejścia ręcznego mają charakter szacunkowy i zostały określone na podstawie doświadczeń zdobytych podczas realizacji projektu oraz analizy poszczególnych etapów procesu. Czasy podejścia zautomatyzowanego pochodzą z logów GitHub Actions i stanowią rzeczywisty pomiar czasu wykonania.

7.3 Czasy wykonania i nakład pracy człowieka

Tabela 4 Porównanie nakładu pracy człowieka i czasu realizacji cyklu

Etap	Ręczny	Zautomatyzowane CI/CD
Przygotowanie i upload danych	~30 min	~10 min
Walidacja danych	~30 min	automatyczna
Trening modelu	~3 h	automatyczny
Ocena jakości i decyzja wdrożeniowa	~15 min	automatyczna
Wdrożenie	~45 min	automatyczne
Nakład człowieka łącznie	~5 h	~20 min
Całkowity czas cyklu	~5 h	~4 h

Wynik oznacza redukcję aktywnego czasu inżyniera o około **93%** (z 300 min do 20 min) na jeden cykl. Jednocześnie całkowity czas cyklu został skrócony o około **20%**, ponieważ głównym składnikiem pozostaje sam trening modelu. Zakładając pewną niedokładność oszacowań, wyniki nadal pozostają jednoznaczne: automatyzacja znacząco redukuje nakład pracy człowieka i skraca czas realizacji procesu.

Kluczowe jest rozróżnienie między **nakładem pracy człowieka** a **całkowitym czasem realizacji**. W podejściu ręcznym obie wartości są praktycznie tożsame, ponieważ operator uczestniczy we wszystkich etapach. W podejściu zautomatyzowanym człowiek dostarcza dane, a następnie pipeline działa bez jego udziału co jest ogromną zaletą dla podejścia zautomatyzowanego.

W praktyce ta różnica może się dodatkowo powiększyć przez fakt, że cykl manualny obejmuje kilkanaście interwencji: uruchomienie środowiska, trening, monitorowanie logów, ocenę metryk, rejestrację modelu, aktualizację konfiguracji wdrożenia i weryfikację działania po deploymentcie. Każdy etap jest potencjalnym źródłem błędów. W pipeline CI/CD większość tych kroków jest wykonywana automatycznie i audytowalnie.

7.4 Jakość modeli i skuteczność bramki jakości

Poniższe wykresy i tabela nr 5 przedstawiają metryki zarejestrowane w MLflow dla kolejnych wersji modelu.



Rys. 31 Metryki jakości modelu w interfejsie MLflow

Tabela 5 Metryki jakości modelu dla kolejnych wersji

Wersja modelu	Dice	IoU	Hausdorff
v21	0,8067	0,7286	49,0034
v20	0,8134	0,7409	59,9987
v18	0,5998	0,5508	226,5247
v16	0,4825	0,4518	94,3321
v14	0,4710	0,4060	181,2840
v13	0,0741	0,0388	148,5599
v12	0,0042	0,0021	358,6141
v11	0,0449	0,0233	355,3105
v10	0,0449	0,0233	355,3105
v9	0,3116	0,2461	329,8586
v8	0,0536	0,0278	336,6613
v7	0,0536	0,0278	336,6613
v5	0,3125	0,1852	272,5949
v3	$3,5689 \cdot 10^{-10}$	$3,5689 \cdot 10^{-10}$	724,0773
v1	$3,5386 \cdot 10^{-10}$	$3,5386 \cdot 10^{-10}$	409,2163

Dane przedstawione w powyższej tabeli wskazują, że wszystkie modele były poprawnie rejestrowane niezależnie od osiągniętych metryk jakościowych.

Tabela 6 Wersje modelu awansowane do etapu Production

Wersja modelu	Dice	IoU	Hausdorff
v20	0,8134	0,7409	59,9987
v18	0,5998	0,5508	226,5247
v16	0,4825	0,4518	94,3321
v14	0,4710	0,4060	181,2840
v9	0,3116	0,2461	329,8586
v5	0,3125	0,1852	272,5949
v3	$3,5689 \cdot 10^{-10}$	$3,5689 \cdot 10^{-10}$	724,0773
v1	$3,5386 \cdot 10^{-10}$	$3,5386 \cdot 10^{-10}$	409,2163



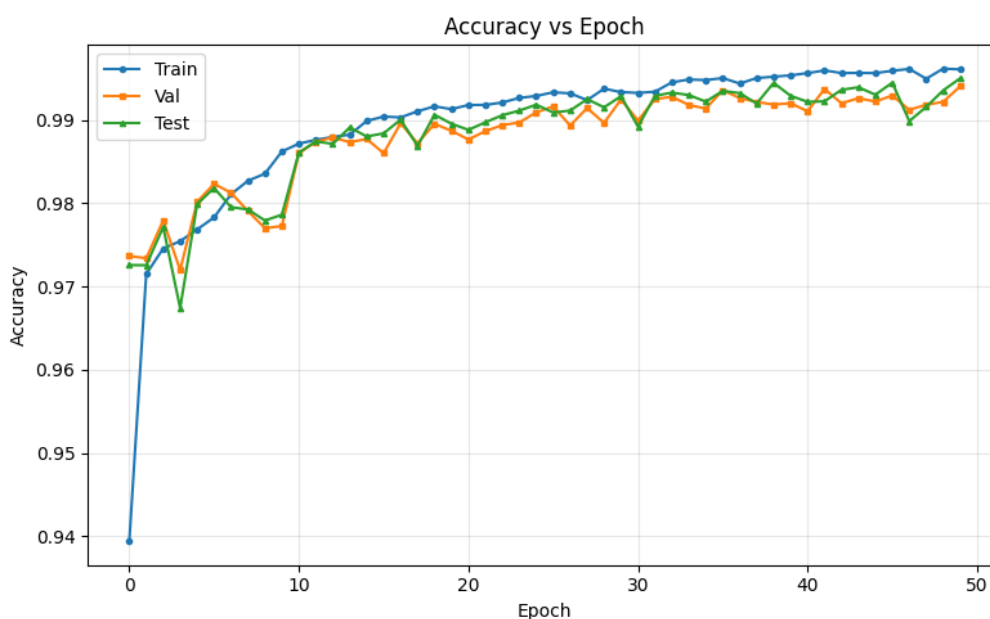
Rys. 32 Rejestr modeli MLflow — widok wersji ze stadium Production

Analiza całego zbioru wersji potwierdza poprawność mechanizmu kontroli jakości. Wszystkie przebiegi zostały zarejestrowane, natomiast do etapu *Production* awansowały wyłącznie wersje spełniające kryteria bramki. Pozostałe wersje nie uczestniczyły we wdrożeniu produkcyjnym.

Warto podkreślić, że bardzo wczesne wersje modelu (v1 i v3) uzyskały wartości Dice/IoU rzędu 10^{-10} , co praktycznie oznacza brak użytecznej zdolności predykcyjnej. Tak skrajne wartości bywają słabo widoczne na wykresach, ponieważ warstwa wizualizacji skaluje osie pod wartości dominujące. Nie jest to błąd eksperymentu ani logowania metryk; wartości są poprawnie zarejestrowane w widoku tabelarycznym MLflow.

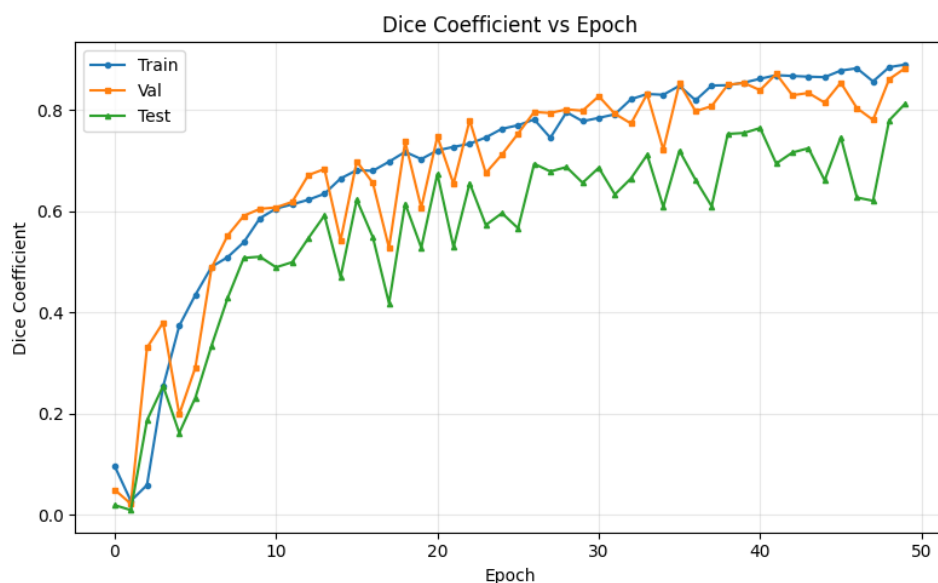
Najlepszy model (v20) osiągnął Dice 0,8134 i IoU 0,7409. Wynik jest niższy od modelu referencyjnego (Dice 0,9275, IoU 0,8865), co wynika ze świadomie przyjętego ograniczenia czasu treningu i budżetu. Celem projektu była walidacja procesu automatycznego utrzymania i kontrolowanej promocji modeli, a nie maksymalizacja metryk absolutnych.

W porównaniu z podejściem manualnym przyjęto konserwatywne założenie, że wszystkie wyniki uzyskane w pipeline CI/CD są osiągalne również ręcznie. Jednocześnie podejście ręczne daje większą swobodę uruchamiania dłuższych treningów i trenowania na większych zbiorach bez narzutu pełnej orkiestracji procesu. Oznacza to, że w wymiarze czysto modelowym (bez uwzględnienia ryzyka operacyjnego i kosztu pracy człowieka) podejście manualne może prowadzić do szybszego dojścia do wysokich metryk, w tym do poziomu zbliżonego do repozytorium referencyjnego przy wykorzystaniu pełnego zbioru danych.



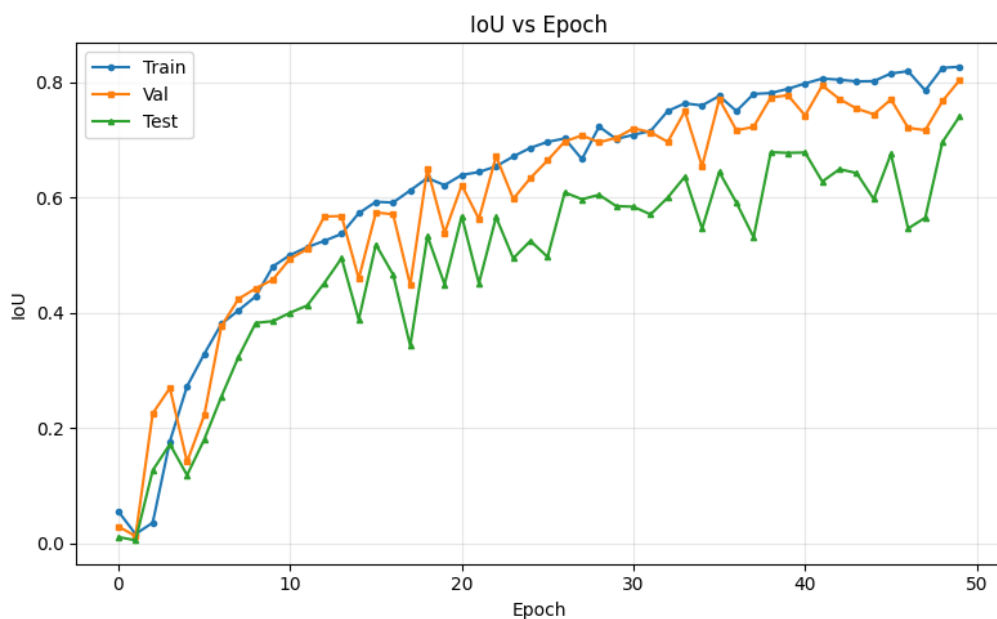
Rys. 33 Wykres metryki Accuracy dla najlepszej wersji modelu (v20)

Metryka Accuracy rośnie szybko i stabilizuje się na poziomie powyżej 0,99 dla wszystkich podzbiorów. W zadaniu segmentacji binarnej jest to jednak metryka pomocnicza, ponieważ może zawyżać ocenę jakości przy dominacji klasy tła.



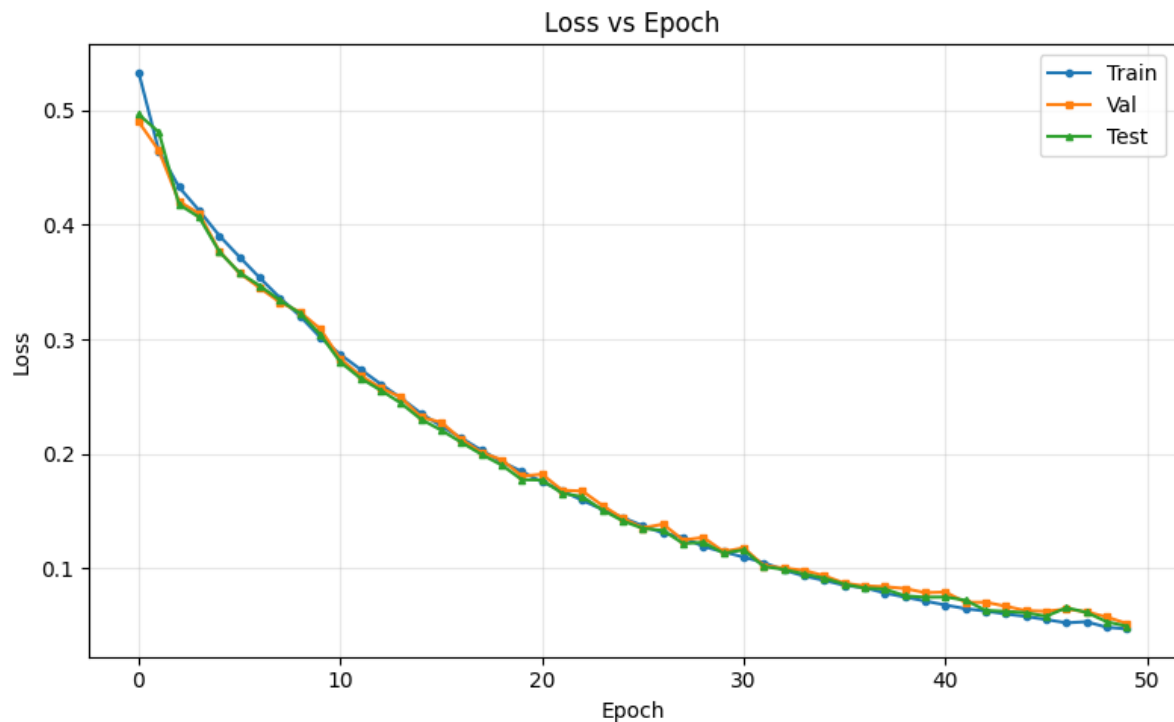
Rys. 34 Wykres metryki Dice dla najlepszej wersji modelu (v20)

Współczynnik Dice, kluczowy dla segmentacji, wykazuje systematyczny wzrost w trakcie uczenia i osiąga wysokie wartości końcowe. Niewielki odstęp między przebiegami treningowym i walidacyjnym wskazuje na dobrą generalizację modelu.



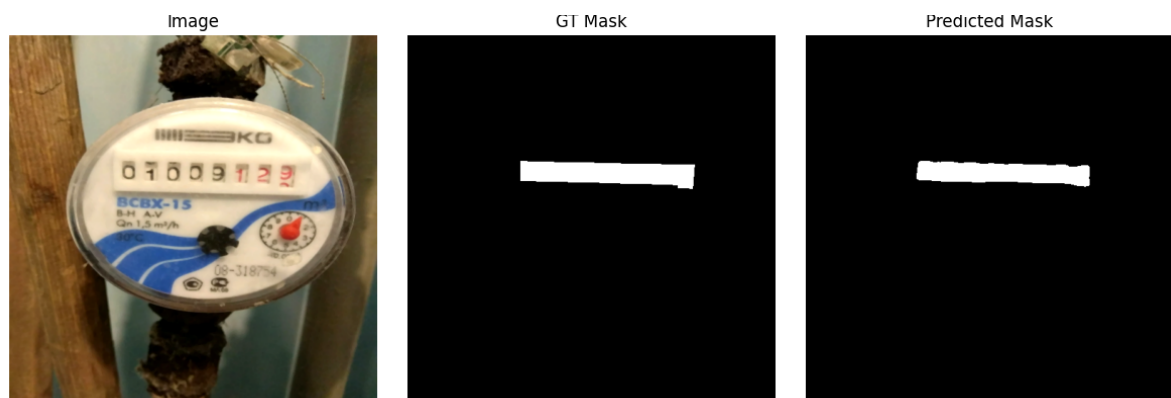
Rys. 35 Wykres metryki IoU dla najlepszej wersji modelu (v20)

Metryka IoU zachowuje trend zgodny z Dice: wyraźnie rośnie wraz z kolejnymi epokami i stabilizuje się pod koniec treningu. Końcowe wartości potwierdzają satysfakcjonującą zgodność predykowanych masek z maskami referencyjnymi.



Rys. 36 Wykres funkcji straty dla najlepszej wersji modelu (v20)

Funkcja straty maleje monotonicznie dla zbioru treningowego, walidacyjnego i testowego, bez oznak niestabilności. Taki przebieg potwierdza poprawną zbieżność procesu uczenia i brak wyraźnych symptomów przeuczenia.



Rys. 37 Przykład predykcji modelu v20 (wejście, maska referencyjna, maska przewidywana)

7.5 Koszty infrastruktury

Analiza kosztowa została zweryfikowana z wykorzystaniem oficjalnego cennika AWS (Price List API) dla regionu `us-east-1`, stan na luty 2026 [1]. Tabela 7 przedstawia stawki jednostkowe użyte do obliczeń.

Tabela 7 Stawki jednostkowe AWS użyte w analizie kosztów

Pozycja	Stawka	Źródło i uwagi
EC2 t3.large Linux On-Demand	\$0,0832/h	AmazonEC2 Price List API, US East (N. Virginia)
EBS gp3	\$0,08/GB-mies.	AmazonEC2 Price List API (wolumen systemowy EC2)
S3 Standard storage	\$0,023/GB-mies.	AmazonS3 Price List API, pierwszy próg do 50 TB
S3 PUT/COPY/POST/LIST	\$0,005/1000 req.	AmazonS3 Price List API
S3 GET/pozostałe	\$0,004/10000 req.	AmazonS3 Price List API
ECR private storage	\$0,10/GB-mies.	AmazonECR Price List API

Do oszacowania kosztu miesięcznego przyjęto realistyczny profil użytkowania środowiska:

- 30 GB wolumenu EBS gp3 dla instancji,
- 20 GB danych i artefaktów w S3,
- 1,5 GB obrazów kontenerowych w ECR,
- 5000 operacji S3 typu PUT/LIST i 30000 operacji GET miesięcznie,
- pojedynczy cykl treningowy: 3,5 h pracy EC2.

Koszt pojedynczego cyklu obliczeniowego wynosi:

$$C_{cykl} = 3,5 \cdot 0,0832 = 0,2912 \text{ USD.}$$

Koszt stały miesięczny (niezależny od liczby cykli) wynosi:

$$C_{stay} = 30 \cdot 0,08 + 20 \cdot 0,023 + 1,5 \cdot 0,10 + \frac{5000}{1000} \cdot 0,005 + \frac{30000}{10000} \cdot 0,004 \approx 3,05 \text{ USD/mies.}$$

Całkowity koszt miesięczny można więc zapisać jako:

$$C_{mies}(N) = 3,05 + 0,2912 \cdot N,$$

gdzie N oznacza liczbę pełnych cykli treningowych w miesiącu.

Tabela 8 Koszt miesięczny dla różnych intensywności użycia pipeline CI/CD

Liczba cykli/mies.	Koszt zmienny EC2	Koszt całkowity
10	\$2,91	\$5,96
30	\$8,74	\$11,79
60	\$17,47	\$20,52

Koszt wdrożenia automatyzacji i porównanie z obsługą manualną (PLN). Do analizy doliczono jednorazowy koszt przygotowania infrastruktury automatycznej:

$$C_0 = 10000 \text{ PLN.}$$

W podejściu manualnym przyjęto, że utrzymanie procesu zajmuje **20% czasu pracy inżyniera** (0,2 etatu). Dla miesięcznej pensji brutto P koszt pracy manualnej wynosi:

$$C_{\text{manual,praca}} = 0,2 \cdot P.$$

Z pomiarów czasu (5 h manualnie vs 20 min w CI/CD) wynika, że obciążenie człowieka po automatyzacji to około 6,67% obciążenia manualnego. Stąd:

$$C_{\text{CI/CD,praca}} \approx 0,0133 \cdot P.$$

Miesięczna różnica kosztu pracy:

$$\Delta C_{\text{praca}} = (0,2 - 0,0133) \cdot P = 0,1867 \cdot P.$$

Czas zwrotu jednorazowego kosztu wdrożenia:

$$T_{\text{BE}} = \frac{C_0}{\Delta C_{\text{praca}}} = \frac{10000}{0,1867 \cdot P} [\text{mies.}].$$

Tabela 9 Próg opłacalności automatyzacji dla różnych pensji miesięcznych

Pensja P [PLN/mies.]	Manual $0,2P$	CI/CD $0,0133P$	Zwrot T_{BE}
8000	1600 PLN/mies.	106 PLN/mies.	6,69 mies.
12000	2400 PLN/mies.	160 PLN/mies.	4,46 mies.
16000	3200 PLN/mies.	213 PLN/mies.	3,35 mies.

Wniosek z tej części analizy: nawet po doliczeniu jednorazowego kosztu 10000 PLN automatyzacja zwraca się względnie szybko względem stałego obciążenia pracy manualnej.

Wyniki pokazują, że automatyzacja CI/CD faktycznie przyspiesza pracę i redukuje nakład operacyjny człowieka, ale odbywa się to kosztem stałych i narastających kosztów chmurowych. W praktyce oznacza to, że podejście CI/CD jest **szybkie i efektywne operacyjnie**, lecz **kosztowne finansowo** przy większej liczbie iteracji oraz przy przejściu do architektury produkcyjnej o wyższych wymaganiach dostępności i bezpieczeństwa.

7.6 Zużycie zasobów i działanie usługi

Serwis inferencyjny działa stabilnie na instancji `t3.large`. Model ładowany jest jednorazowo przy starcie kontenera, a monitorowane metryki API (liczba predykcji, opóźnienia, błędy) potwierdzają poprawne działanie wdrożenia. Dla przyjętej skali obciążenia zasoby CPU i RAM były wystarczające.

W kontekście obciążenia należy podkreślić, że projekt był testowany w skali laboratoryjnej, a nie pod pełnym ruchem produkcyjnym. Oznacza to, że sekcja zasobowa potwierdza wystarczalność środowiska dla przyjętego scenariusza, ale nie stanowi pełnego wyznacznika wydajnościowego dla skali wielokrotnie większej.

7.7 Ocena jakościowa procesu

7.7.1 Łatwość wdrożenia

Konfiguracja systemu wymagała jednorazowego, istotnego nakładu pracy: przygotowania Terraform, konfiguracji k3s i Helm, integracji MLflow z S3, implementacji walidacji danych i bramki jakości oraz przygotowania workflow GitHub Actions. Po wykonaniu tej inwestycji wejściowej kolejne iteracje procesu są realizowane z minimalną interwencją operatora.

Podejście ręczne nie ma kosztu wejścia na poziomie automatyzacji, ale generuje koszt powtarzalny przy każdym cyklu. Wraz ze wzrostem liczby iteracji rośnie obciążenie inżyniera oraz ryzyko błędu ludzkiego.

7.7.2 Stabilność i podatność na błąd

W podejściu zautomatyzowanym trzy elementy istotnie ograniczają ryzyko błędu ludzkiego:

- automatyczna walidacja danych wejściowych przed treningiem,
- automatyczna bramka jakości blokująca regresję modelu,
- automatyczne zatrzymanie EC2 po zakończeniu pipeline'u (także po błędzie).

W podejściu ręcznym wszystkie trzy decyzje pozostają po stronie operatora, co zwiększa prawdopodobieństwo przeoczeń i kosztownych pomyłek.

Tabela 10 Porównanie ryzyka operacyjnego: manual vs CI/CD

Obszar ryzyka	Podejście ręczne	CI/CD
Walidacja danych	zależna od dyscypliny operatora, ryzyko pominięcia błędnych próbek	automatyczna bramka QA blokująca dalszy pipeline
Decyzja o promocji modelu	ręczna interpretacja metryk, podatność na presję czasu	obiektywne kryterium bramki jakości (Dice/IoU)
Koszt chmury po awarii procesu	ryzyko pozostawienia uruchomionej instancji	automatyczne stop-infra po błędzie (always())
Odtwarzalność eksperymentu	wymaga ręcznej dokumentacji i dyscypliny	logi workflow + MLflow + DVC zapewniają ślad audytowy

7.7.3 Skalowalność

Najważniejsza przewaga automatyzacji dotyczy skalowalności pracy zespołu: wzrost liczby iteracji nie zwiększa nakładu pracy człowieka proporcjonalnie do liczby cykli. Ograniczenia obecnej implementacji (single-node k3s, jeden runner, środowisko AWS Academy) mają charakter infrastrukturalny i budżetowy, nie logiczny. Logika pipeline'u może być przeniesiona do większego środowiska bez zmiany modelu procesu.

Tabela 11 Skalowalność nakładu pracy człowieka przy wzroście liczby cykli

Liczba cykli / mies.	Manual (5 h/cykl)	CI/CD (20 min/cykl)
10	50 h	3 h 20 min
30	150 h	10 h
60	300 h	20 h

Zestawienie to pokazuje, że przy wzroście liczby iteracji różnica obciążenia człowieka rośnie liniowo i szybko staje się kluczowym argumentem organizacyjnym na rzecz automatyzacji.

8 Podsumowanie i wnioski

8.1 Osiągnięcia projektu

W ramach niniejszej pracy zaprojektowano, wdrożono i przetestowano kompletny, zautomatyzowany pipeline CI/CD dla modeli uczenia maszynowego. System prowadził model przez każdy etap cyklu życia, poczynając od dostarczenia danych przy użyciu `git push`, przez automatyczną walidację i scalanie zbiorów, trening na instancji EC2, ocenę jakości za pomocą bramki porównującej nowy model z aktualną wersją produkcyjną, a kończąc na wdrożeniu API i automatycznym scaleniu zmian z gałęzią główną repozytorium. Infrastruktura AWS zdefiniowana jest jako kod w Terraform, wdrożenia na klastrze k3s zarządzane są przez Helm, a pełny ślad audytowy każdego eksperymentu przechowywany jest w MLflow.

Zarejestrowano 15 wersji modelu, z których 8 awansowało do etapu Production na podstawie obiektywnych kryteriów bramki jakości. Najlepsza wersja (v20) osiągnęła Dice 0,8134 i IoU 0,7409, wyraźnie przekraczając poziom użytecznej segmentacji. Wszystkie wersje były rejestrowane niezależnie od uzyskanych metryk, co potwierdziło poprawność i niezawodność mechanizmu logowania eksperymentów.

8.2 Odpowiedź na pytania badawcze

W rozdziale wprowadzającym postawiono dwa pytania badawcze. Pierwsze dotyczyło wykonalności i sensowności ekonomicznej automatycznego generowania powtarzalnych fragmentów kodu przy użyciu modelu AI. Drugie pytanie koncentrowało się na tym, czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymania takiego rozwiązania, czy jego zautomatyzowanie z użyciem narzędzi DevOps i procesów CI/CD.

W odniesieniu do pierwszego pytania należy podkreślić, że projekt z założenia weryfikował wykonywalność procesu automatyzacji, a nie samą opłacalność modelu AI w konkretnej domenie. Zastępczy problem segmentacji semantycznej, wybrany ze względu na jednoznacznie mierzalne metryki jakości oraz umiarkowane wymagania obliczeniowe, potwierdził techniczną wykonalność pełnej automatyzacji cyklu życia modelu AI. System samodzielnie zarządzał kolejnymi wersjami modelu, egzekwował obiektywne kryteria jakości oraz utrzymywał spójność między kodem, danymi i wdrożeniem. Wykazuje to, że zarówno infrastruktura techniczna, jak i logika procesu są wystarczająco dojrzałe, aby zastąpić ręczną orkiestrację, nawet przy ograniczonym budżecie i w środowisku laboratoryjnym.

Odpowiedź na drugie pytanie jest jednoznaczna. Automatyzacja jest operacyjnie opłacalna przy regularnych iteracjach. Aktywny czas inżyniera spadł z około 5 godzin do około 20 minut na cykl, co stanowi redukcję o 93%. Analiza progu zwrotu wskazuje, że przy założeniu jednorazowego kosztu wdrożenia na poziomie 10 000 PLN inwestycja zwraca się w 3–7 miesięcy w zależności od wynagrodzenia inżyniera, a przy 30 cyklach miesięcznie szacowane obciążenie manualnego utrzymania wynosiłoby ponad 150 roboczogodzin miesięcznie wobec 10 w podejściu zautomatyzowanym. Dane te pokazują, że kosztem jednorazowej inwestycji i stałych opłat chmurowych eliminuje się koszt powtarzalny, który rośnie liniowo wraz z liczbą iteracji.

8.3 Kluczowe wnioski z porównania

Zasadnicza różnica między podejściami nie leży wyłącznie w czasie wykonania, lecz w charakterze zaangażowania człowieka. W podejściu ręcznym operator uczestniczy we wszystkich etapach procesu, obejmujących uruchomienie środowiska, monitorowanie treningu, interpretację metryk, rejestrację modelu, aktualizację konfiguracji wdrożenia i weryfikację działania API. Każda z tych interwencji jest potencjalnym źródłem błędu lub niespójności. W pipeline CI/CD człowiek wykonuje wyłącznie jedno działanie i dostarcza dane, a cały pozostały przepływ jest deterministyczny, audytowalny i niezależny od stanu wiedzy czy dyspozycyjności konkretnej osoby w danym momencie.

Automatyzacja gwarantuje ponadto powtarzalność eksperymentów niemożliwą do osiągnięcia manualnie przy tej samej skali iteracji. Połączenie DVC z MLflow tworzy pełny ślad audytowy pozwalający odtworzyć dowolny eksperyment w identycznych warunkach, podczas gdy w podejściu ręcznym odtwarzalność zależy od dyscypliny dokumentacyjnej operatora i jest trudna do wyegzekwowania systemowo.

Podejście manualne zachowuje przewagę elastyczności w fazie eksploracyjnej. Nie wymaga wstępnej inwestycji i pozwala swobodnie uruchamiać niestandardowe eksperymenty bez dopasowywania ich do logiki pipeline. Jednak wraz ze wzrostem liczby iteracji koszt tej elastyczności zaczyna dominować nad jej zaletami.

8.4 Ograniczenia badania

Wyniki należy interpretować z uwzględnieniem kilku istotnych ograniczeń. Czasy dla podejścia ręcznego są szacunkowe, wyznaczone na podstawie doświadczeń zdobytych podczas realizacji projektu, a nie bezpośrednich pomiarów porównawczego eksperymentu. Szacunki mają charakter konserwatywny, jednak zmienność wynikająca z poziomu doświadczenia inżyniera oraz warunków konkretnego dnia pracy pozostaje niemierzalna.

Pomiar nakładu pracy dla podejścia zautomatyzowanego (około 20 minut na cykl) obejmuje wyłącznie operacyjny koszt dostarczenia danych i uruchomienia pipeline. Nie uwzględnia czasu poświęcanego na utrzymanie samej infrastruktury automatyzacji, w tym ręcznej rotacji kluczy sesyjnych AWS, diagnostyki przerywanych kroków workflow oraz aktualizacji zależności. Ograniczenia środowiska AWS Academy, obejmujące budżet 50 USD i brak uprawnień do konfiguracji OIDC, wymuszały ponadto uproszczone wybory architektoniczne. Środowisko produkcyjne z OIDC i wielowęzłowym klastrem miałoby inną strukturę kosztów, a analiza dotyczy wyłącznie bieżącej konfiguracji.

Problem zastępczy (segmentacja wodomierzy zamiast generowania kodu testowego) był wyborem celowym i uzasadnionym, ale oznacza, że wyniki dotyczące jakości modelu nie przekładają się bezpośrednio na domenę docelową. Celem było zbadanie procesu, a nie osiągnięcie konkretnej jakości modelu, co jest w pracy konsekwentnie zaznaczone. Infrastruktura single-node k3s z jednym runnerem, choć wystarczająca dla celów demonstracyjnych, nie odzwierciedla skali ani odporności środowiska produkcyjnego.

8.5 Wnioski końcowe

Niniejsza praca postawiła pytanie, czy automatyzacja DevOps/CI/CD jest uzasadniona w kontekście utrzymania modeli sztucznej inteligencji. Odpowiedź udzielona przez zrealizowany projekt jest twierdząca, choć zależy od kontekstu użycia.

Technicznie automatyzacja jest uzasadniona, ponieważ system działa poprawnie, egzekwuje obiektywne kryteria jakości i zarządza kolejnymi wersjami modelu bez interwencji człowieka. Operacyjnie argumenty na jej rzecz rosną wraz z dojrzałością produktu, a redukcja nakładu pracy o 93% i liniowa skalowalność względem liczby iteracji są trudne do podważenia. Nie każdy scenariusz uzasadnia jednak tę inwestycję. Przy niskiej liczbie iteracji lub w fazie intensywnej eksploracji modelu koszt wdrożenia infrastruktury może przewyższać uzyskane oszczędności, a elastyczność podejścia manualnego zachowuje wówczas rzeczywistą wartość.

Kluczowym wnioskiem metodologicznym jest to, że wartość automatyzacji ujawnia się w czasie, a nie w pojedynczym cyklu. Pierwszy deployment manualny może być szybszy od pierwszego deploymentu zautomatyzowanego. Jednak przy dziesiątym, trzydziestym czy sześćdziesiątym cyklu proporcje radykalnie się odwracają. Z perspektywy organizacyjnej oznacza to, że decyzja o wdrożeniu pipeline CI/CD powinna być podejmowana na etapie planowania projektu, a nie jako reakcja na rosnące obciążenie manualne, bo retroaktywna automatyzacja jest kosztowniejsza niż zaprojektowanie procesu poprawnie od początku.

Zrealizowany projekt potwierdza, że techniki DevOps, wypracowane pierwotnie dla klasycznego oprogramowania, przenoszą się skutecznie na obszar uczenia maszynowego. Różnice między kodem aplikacji a modelem AI jako artefaktem wdrożeniowym są realne i wymagają dostosowań takich jak wersjonowanie danych, śledzenie metryk czy dynamiczny baseline dla bramki jakości. Jednak fundamentalne zasady pozostają niezmienione. Powtarzalność, automatyczna kontrola jakości, audytowalność i infrastruktura jako kod przynoszą te same korzyści niezależnie od tego, czy wdrażanym artefaktem jest biblioteka oprogramowania czy wagi sieci neuronowej. W miarę jak modele AI stają się coraz powszechniejszym elementem systemów produkcyjnych, umiejętność zautomatyzowanego zarządzania ich cyklem życia przestaje być przewagą konkurencyjną, a staje się podstawowym wymogiem projektowania oprogramowania.

8.6 Wykorzystanie narzędzi sztucznej inteligencji

W trakcie realizacji pracy korzystano z narzędzi opartych na modelach językowych (Claude Code, ChatGPT, Gemini) jako wsparcia przy implementacji, diagnostyce oraz redakcji treści. Narzędzia te pełniły rolę asystującą. Ostateczne decyzje projektowe, implementacyjne oraz interpretacyjne pozostawały po stronie autora.

Literatura

- [1] Amazon Web Services. AWS pricing — oficjalne cenniki usług chmurowych. <https://aws.amazon.com/pricing/>, 2026. Dostęp: 24.02.2026.
- [2] Bluemetrica. Czym jest mlops? <https://bluemetrica.com/czym-jest-mlops/>, 2024. Dostęp: 16.02.2026.
- [3] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016. doi: 10.1145/2898442.2898444. <https://dl.acm.org/doi/10.1145/2898442.2898444>.
- [4] Scott Chacon and Ben Straub. Pro git. <https://git-scm.com/book>, 2014. Dostęp: 10.02.2026.
- [5] DataFight. U-net i segmentacja obrazu. <https://datafight.wordpress.com/2020/05/04/u-net-i-segmentacja-obrazu/>, 2020. Dostęp: 10.02.2026.
- [6] GitHub Inc. Github actions documentation. <https://docs.github.com/en/actions>, 2024. Dostęp: 10.02.2026.
- [7] HashiCorp. Terraform documentation. <https://developer.hashicorp.com/terraform/docs>, 2024. Dostęp: 20.02.2026.
- [8] Helm Authors. Helm — the kubernetes package manager. <https://helm.sh/docs/>, 2023. Dostęp: 10.02.2026.
- [9] Iterative.ai. Data version control — open-source version control system for machine learning projects. <https://dvc.org/doc>, 2023. Dostęp: 10.02.2026.
- [10] Kaggle. Yandex toloka water meters dataset. <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>, 2023. Dostęp: 10.02.2026.
- [11] Grafana Labs. Grafana documentation: Introduction. <https://grafana.com/docs/grafana/latest/fundamentals/>, 2024. Dostęp: 16.02.2026.
- [12] Mike Loukides. What is devops? <https://cdn.bookey.app/files/pdf/book/en/what-is-devops-.pdf>, 2012. Bookey summary PDF, Dostęp: 16.02.2026.
- [13] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.
- [14] Nehal Navlani. Ci/cd pipeline - system design. <https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/>, 2025. Dostęp: 10.02.2026.
- [15] MLflow Project. Mlflow model registry. <https://mlflow.org/docs/latest/ml/model-registry/>, 2024. Dostęp: 16.02.2026.

- [16] Prometheus Authors. Prometheus — monitoring system and time series database. <https://prometheus.io/docs/introduction/overview/>, 2023. Dostęp: 10.02.2026.
- [17] Rancher Labs. k3s — lightweight kubernetes. <https://docs.k3s.io>, 2023. Dostęp: 10.02.2026.